Part -01 : Java basics, control statements, standard programs and pattern programs[20 classes]
Start date : 16th July
End date : 8th august

What is java ?
Java is a programming language used to build applications and software.

History :
1995 : james Gosling→ Father of java.
Sun Microsystems : US based company.
GreenFoot➔Oak➔Java
Java➔an island name in Indonesia
Java➔ coffee brand
Java alpha version(trail version)
Java beta version(trail version)
Java 1.0
Java 1.2
Java 1.3
Java 1.4
Java 1.5
Java 1.6
Java 1.7
Java 1.8
sunMicrosystems➔1995 to 2009
Oracle ➔ 2010 onwards, till now

2017 - September ➔ java 9
Note : every year oracle will release two versions of java.
Even versions ➔march
Odd versions ➔ September
2018 – march →java 10
2018 – September→ java 11
.
.
.
2026 – march→java 26

CodeWithMahi

Characteristics of java :

1)java is object oriented programming language
    Java supports OOPS concepts.
    Major 4 pillars of oops
        1)inheritance
        2)encapsulation
        3)polymorphism
        4)abstraction
2)java is a case sensitive language.
3)java is a syntax based language.
4)java is platform independent language.

Different operation stages in java :

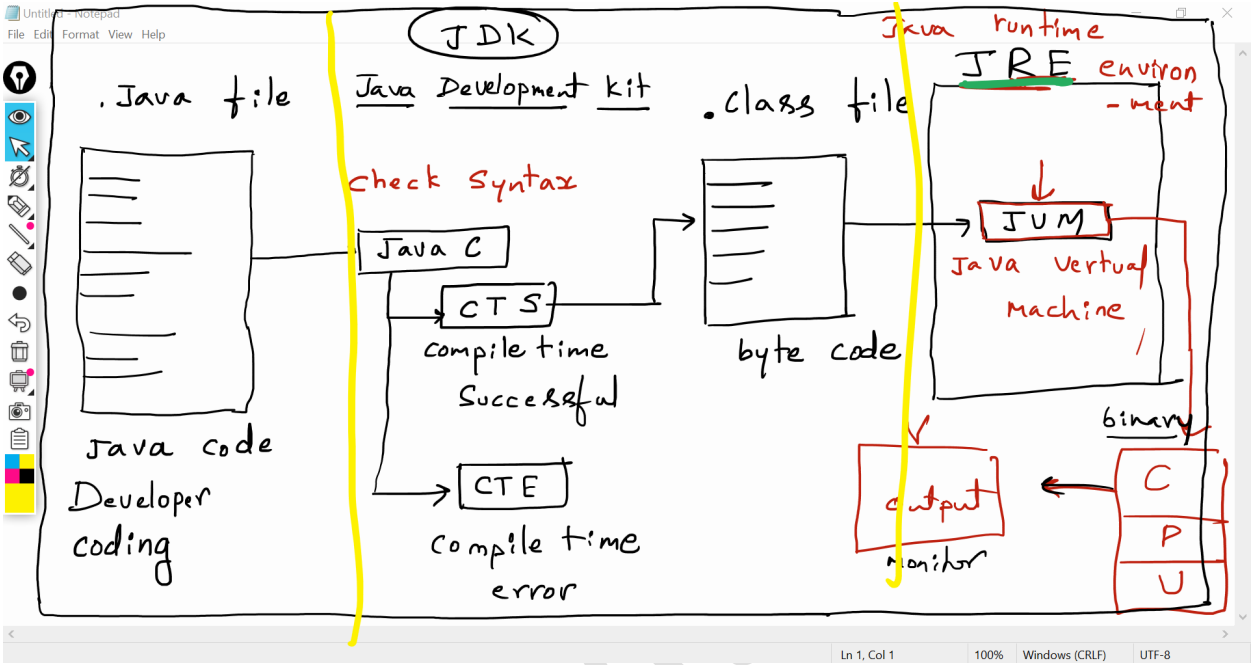
1)coding
2)compilation
3)execution

1)coding
It is the process of writing java instructions in a file.
Responsible : Developer
File : java file/ .java file/ src file

2)compilation :
In this stage, java code will be converted to byte code.
Responsible : compiler
File : class file / .class file

3)Execution :
In this stage, byte code(.class file) will be executed or performed.
Responsible : JVM
File : No files created.

CodeWithMahi




```
Basic structure of java program :
Before java 25 :
class className
{
      public static void main(String [] args)
      {

      }
}
```

public : it is a access modifier.(public, protected, default and private)
static : it is a keyword used to mark the java components as static.
void : it is the return type of a method.
String [] args : it is a String array.
It is used as argument for the main method.

Printing statement :

System.out.println("data");
System → is a inbuilt class present in java.lang package.

out→it is a static object reference variable created to PrintStream class.
println()→it is a non-static, overloaded method present in PrintStream class.
From java 25 :

```
class className
{
      void main()
      {

      }
}
```

Printing statement :

```
IO.println("data");
```

How to write and execute java code in note pad.

1)write java code in note pad and save the file as className.java in the type "All Files".
2)go the location of the file, click on address bar and type cmd.
It will open the command prompt.
3)for compiling the file use the command : javac className.java then click on enter.
4)after the 3rd step, you will see a class file created in the same location.
5)later, to execute the code, use the command : java className and click on enter to see the output.

Types of editors :

1)basic editor :

Example : Notepad, notepad++, editplus.
These basic editors will not provide shortcuts to write the code.
These editors will not provide suggestions while writing the code.
Everything, developer has to type.
It makes hard for developer to write the code.

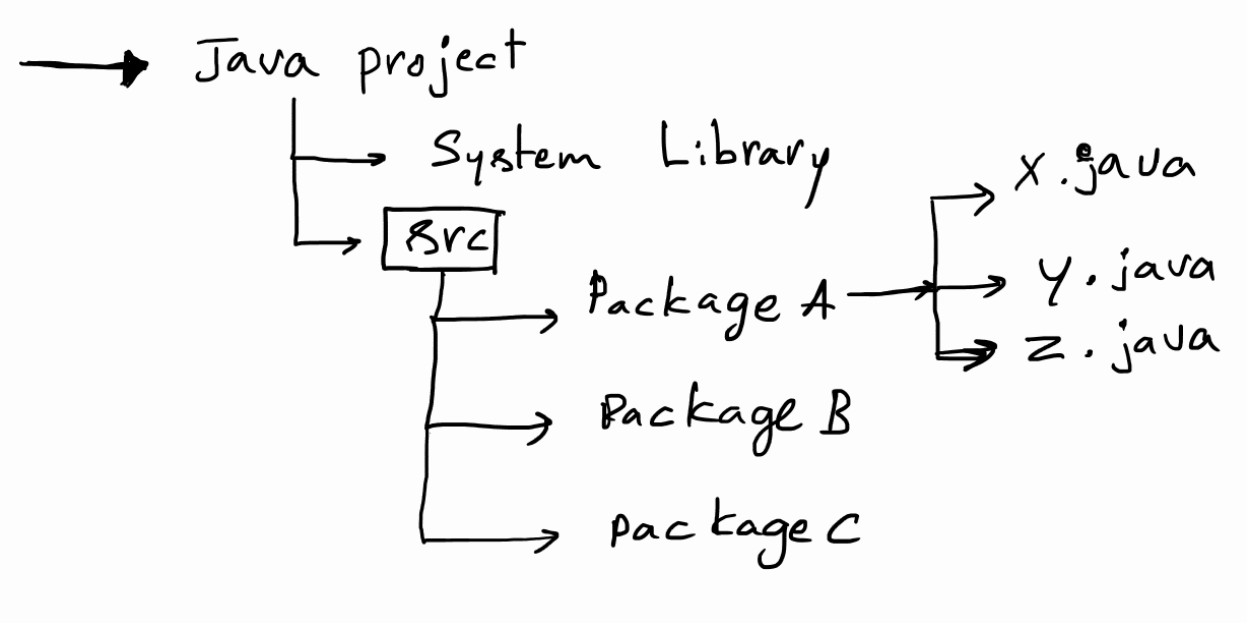
2)advance editor :

Example : eclipse, IntelliJ, VScode, etc….

CodeWithMahi

These advanced editors will provide shortcuts to write the code.
These editors will provide suggestions while writing the code.
Not Everything should be written by a developer.
It makes easy for developer to write the code.

Writing code in eclipse :



Types of casing :
1)Pascal casing
2)camel casing

1)Pascal casing : should start with capital letter
Rules :
1)First letter of each word should be capital.
2)No space given in the sentence.
3)Sentence can have only _ and $ symbols.
4)Sentence should not start with digits.
5)Sentence can contain digits.

1)camel casing : should start with small letter
Rules :
1)First letter of each word should be capital.
2)No space given in the sentence.

3)Sentence can have only _ and $ symbols.
4)Sentence should not start with digits.
5)Sentence can contain digits.
Example : Dog loves to play
Pascal casing : DogLovesToPlay
camel casing : dogLovesToPlay

➔project name follows Pascal casing
➔package name follows camel casing
➔class name follows Pascal casing

Types of java

1.java SE : java standard edition

This is used to build desktop applications and command line softwares. Basically it is used in system development.

2.java EE : java entrepreneur edition

this involves advance java concepts like jdbc, servlets, etc.. which are majorly used in database connections and deployment related tasks.

3.java ME : java micro edition

This is an advanced edition of java EE. This is majorly used in small computer designings like old phones, etc…..

4.Java card :

this version of java is majorly used in building smart cards like, sim card, atm cards, etc…..

5. java FX :

This version of java is majorly used in building some of the applications where they use heavy animations in building apps and softwares.

Java tokens : [CISKO-L]

C ➔Comments : this format is used to exclude the part of code from the actual code or top provide information about the code.

1)Single – line comment :

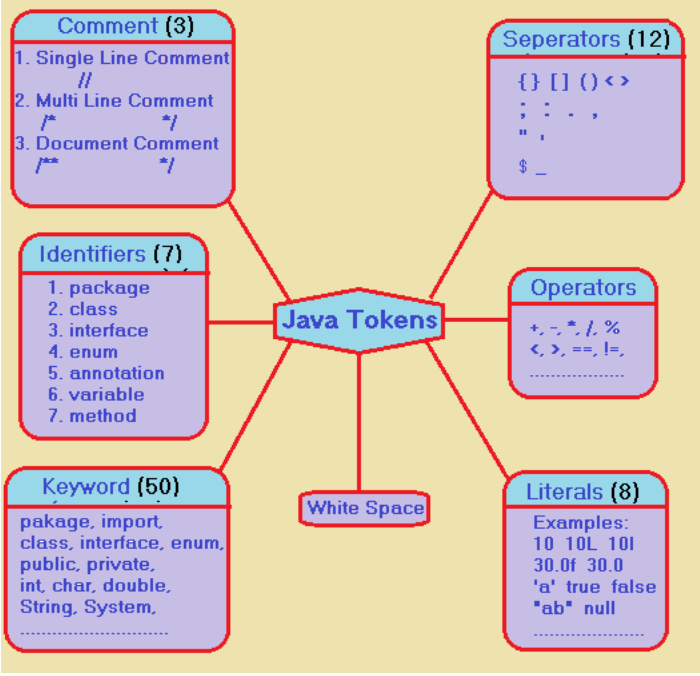
It is used to comment a single line.

CodeWithMahi

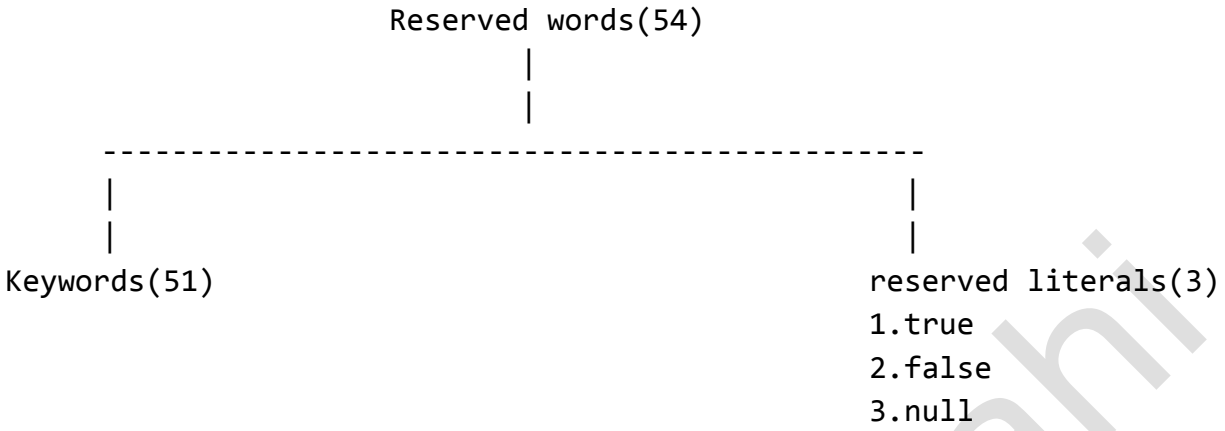
Syntax : //statement
Shortcut : ctrl+/
2)multi-line comments : used to comment multiple lines at a time.
Syntax : /*…….*/
3)Document comment : used to explain a whole document.
These are used in in-built classes/interfaces.
Syntax : /**…….*/

I➔ identifiers :
These are the names provided by developer to the java components like, Project name, package name, class name, method name, variable name, etc…..
➔identifiers have to follow Pascal or camel casing.

Separators :
These are the symbols used to separate two different parts of the java components.



K- Keywords :
These are the pre-defined words which have specific meaning in java. Used for respective tasks.



CodeWithMahi

Keywords(51)

Used keywords(49)

un-used keywords(2)
1.goto
2.const



O-Operators :
These are the symbols used to perform a specific operations.
Example : +,-,*,>,<,!=,==, etc…..

L-Literals :
These are the data that we use while coding.
11,true,false, 4.5,etc……..

Datatypes :
Datatypes are the keywords used to represent the type of data.
Example : int, float, char, etc…..

Two types :
1)primitive datatypes
2)non-primitive datatypes

Primitive datatypes(8):

Sl. number	*Datatype*	*Type of the data it represents*	*Memory size*	*Range*
1	byte	Integers	1 byte	-2^7 to 2^7-1 -128 to 127
2	short	Integers	2 bytes	-2^{15} to $2^{15}-1$ 32768 to 32767
3	int (default)	Integers	4 bytes	-2^{31} to $2^{31}-1$ 2147483648 to 2147483647
4	long	Integers	8 bytes	-
5	float	Decimal point numbers	4 bytes	-
6	double (default)	Decimal point numbers	8 bytes	-
7	boolean	true / false	1 bit	-
8	char	single character	2 bytes	-

➔primitive datatypes are only pre-defined.
➔we cannot create primitive datatypes.
➔we cannot create objects for primitive datatypes.
Because of this reason, we say **"java is not fully object oriented programming language".**
➔primitive datatypes always starts with lowercase.
byte→ primitive datatype
Byte→ non-primitive datatype.
➔for long type of data, we have to write letter l at the last.
➔for float type of data, we have to write letter f at the last.
➔char data is written in the single codes.
➔String data should be written in double codes.
➔String is a group of characters.

Non-primitive datatypes :
These are the classes which are pre-defined or user-defined.
Example : user-defined datatypes
class Animal{}
class Student{}
class Vehicle{}
here, the classes like, Animal, Student, Vehicle, etc are non-primitive datatypes.
Example : pre-defined datatypes
String, StringBuilder, StringBuffer, System, etc……

➔We also can create non-primitive datatypes.
➔every class is a non-primitive datatype.
➔we can create objects for non-primitive datatypes.
➔non-primitive datatypes starts with capital letter.
➔except 8 primitive datatypes, all are non-privitive datatypes.

Variable : Variable is a memory block used to store the data.
Syntax : datatype variableName = data ;
1)**Variable declaration** : it is the process of reserving a small piece of memory.
Syntax : datatype variableName;

2)**Variable initialization** : it is the process of initializing the declared variable.
Syntax : variableName = data;

Note : **variable names follow camel casing.**
Example :

```
package basicsOfJava;

public class VariablesDemo
{
	public static void main(String[] args)
	{
		//variable declaration
		//int age;
```

CodeWithMahi

```
            //variable initialization
            //age = 32;

            //variable declaration and initialization
            int age = 32;
            System.out.println(age);
      }
}
```

What is re-initialization?

➔storing the data into a variable which is already initialized.

Example :

```
int age = 32;
age = 40;
```

➔When you do re-initialization, don't write data type again.

➔we cannot create two or more variables with the same name.

Example :

```
//variable re-initialization
int age = 32;
age = age + 20;
System.out.println(age);
```

➔for variable names don't write single or double codes.

Primitive-Datatype Casting :

It is the process of converting one type of data to another type.

2 types :

1)Widening datatype casting :

2)Narrowing datatype casting :

1)Widening datatype casting :

It is the process of converting Lower memory datatype to higher memory datatype.

Syntax : (Higher Memory DataType) varaibleName;

➔widening datatype casting can be done both implicitly and explicitly.

➔Why implicitly? : because we are converting lower memory datatype to higher memory datatype. Hence we don't face memory shortage. So

CodeWithMahi

compiler accepts that there is no chance of losing the data and compiler will do datatype casting in the absence of developer.

2) Narrowing datatype casting :
It is the process of converting Higher memory datatype to lower memory datatype.
Syntax : (lower Memory DataType) varaibleName;
➔Narrowing datatype casting can be **only done** explicitly.
➔Why only explicitly? : because we are converting higher memory datatype to lower memory datatype. Hence we face memory shortage. So compiler wont accept. there is chance of losing the data and compiler will not do datatype casting in the absence of developer.

➔Integer values can be stored in decimal point datatypes but vice-versa not possible.

ASCII values : **American standard code for information interchange.**
➔for every character there will be a unique universal integer value called ASCII values.

'0' – '9' ➔ 48 to 57
'A' – 'Z' ➔ 65 to 90
'a' – 'z' ➔ 97 to 122

Operators in java :
Operator : operator is a symbol used to perform specific operation as pre-defined.
Example : +,-,*,/,%,>,<,>=,<=,==,=,!=,!,&&,||, etc…….

Operand : it is the data that we use to perform the operation.
Example : 10, true, 'c', "Hello", etc…..{all literals}

Example : 5+4-6
Here, 5,4 and 6 are operands
+ and – are operators.

Types of operators [depending on number of operands]:

CodeWithMahi

1)**Unary operator** : these operators need only one operand to perform the operation.
Example : ++,--,!, etc…..
2)**Binary operator** : these operators need two operands to perform the operation.
Example : +,-,*,/, >, etc……..

3)**Ternary operator** : these operators need more than two operands to perform the operation.
Example : Conditional operator : [condition ? expression-01 : expression-2 ;]

Operators in java : [ARLA-CIC-B.Sc]
1)Arithmetic operators➔{+, -, *, /, %}
2)Relational operators➔{>,>=,<,<=,==,!=}
3)logical operators➔{AND(&&), OR(||), NOT(!)}
4)Assignment operator➔{=}
5)conditional operator➔{ condition ? expression-01 : expression-2 ;}
6)increment/decrement operators{++,--}
7)combinational operators{+=, -=, *=, /=, %=}
8)bitwise operators
9)shift operators{<<, >>}

1)Arithmetic operators➔{+, -, *, /, %}
+ → addition ➔ System.out.println(4+5); // 9
- → subtraction ➔ System.out.println(5-4);//1
* → multiplication ➔System.out.println(5*2);//10
/ →division(quotient)➔System.out.println(17/5);//3
% → division(remainder) ➔System.out.println(17%5);//2

```
System.out.println(17.0/5);//3.4
System.out.println(17.0%5);//2.0
```

int / int ➔ int
double / int (or) int / double (or) double / double ➔ double

Datatype 1	*Datatype 2*	*Result*
byte	**byte**	**int**
short	**short**	**int**
int	**int**	**int**

CodeWithMani

long	long	long
float	float	double
double	double	double
int	char	int
double	char	double
char	char	int

String concatenation :

Adding or joining of any type of data[int, double, boolean, char, String] with a String is called concatenation.

➔only + operator will perform concatenation.

➔compulsory one operand must be a String.

```
package basicsOfJava;

public class VariablesDemo
{
	public static void main(String[] args)
	{
		System.out.println(4+5);//9
		System.out.println(10+'A');//75
		System.out.println('A'+'B');//131
		//System.out.println(false+true);//CTE
		//System.out.println(false-false);//CTE
		//System.out.println(10-true);//CTE
		System.out.println(10*'A');//650
		//System.out.println(10*false);//CTE
		System.out.println('B'-20);//46
		System.out.println("Hello"+10);//Hello10
		System.out.println("Hello"+10+20);//Hello1020
		System.out.println("Hello"+(10+20));//Hello30
		System.out.println(10+20+"Hello");//30Hello
		System.out.println(10+"Hello");//10Hello
		System.out.println("Chicken"+'A');//ChickenA
		System.out.println("Hello"+true);//Hellotrue
		//System.out.println("Hello"-10);//CTE
		//System.out.println('A'+true);//CTE
		System.out.println("Hello "+"everyone!!");//Hello everyone
		//System.out.println(false-false);//CTE
	}
}
```

Relational operators :

```
> ➔ System.out.println(5>4);    // true
< ➔ System.out.pritnln(4<3);    //false
>= ➔ System.out.println(6>=5);  //true
```

CodeWithMahi

<= ➔ System.out.println(10<=5); //false
== ➔ System.out.println(5==5); //true
!= ➔ System.out.println(5!=5); //false

Logical operators :
1)**And** ➔ && ➔ if both inputs are true, then result is true.

Input-01	Input-02	Output
true	false	false
false	true	false
true	true	true
false	false	false

2)**Or** ➔ || ➔ if both inputs are false, then result is false.

Input-01	Input-02	Output
true	false	true
false	true	true
true	true	true
false	false	false

3)Not ➔! ➔ opposite result.

Input	Output
true	false
false	true

Assignment operator :
= is the assignment operator used to assign the values to the variables from RHS to LHS.

int a = 10;
here, 10 is assigned to variable a.

Example:

```
int a = 10;
int b = 20;
a = b;
System.out.println(a);
System.out.println(b);
```

CodeMahMani

Example :

```
int a = 10;
int b = 20;
a = (a+b);
System.out.println(a);
System.out.println(b);
```

Conditional operator :

Syntax : (condition) ? expression-01 : expression-02;

Example :

```
int a = 40;
int b = (a<20) ? 50 : 100 ;
System.out.println(b);
```

Q1)WAPTP larger value between two variables.

```
package basicsOfJava;

public class LargerBetweenTwo
{
	public static void main(String[] args)
	{
		int a = 20;
		int b = 15;

		int largest = (a>b) ? a : b ;

		System.out.println("Largest value is : "+largest);
	}
}
```

Q2)WAPTP largest value among three variables.

```
package basicsOfJava;

public class LargestAmongThree
{
	public static void main(String[] args)
	{
		int a = 25;
		int b = 40;
		int c = 100;
```

CodeWithMahi

```
            int largest = (a>b && a>c) ? a : (b>c) ? b : c ;
            System.out.println("Largest element is : "+largest);
      }
}
```

Assignment Question-3 : WAPTP largest among the four variables using conditional operator/ternary operator.

Q4)WAPTP a number is even or odd using conditional operator.
Input : 3;
Output : odd number
Input : 4
Output : Even number

```
package basicsOfJava;
public class EvenOrOdd
{
      public static void main(String[] args)
      {
            int a = 4;
            String result = (a%2==0) ? "Even Number" : "Odd number" ;
            System.out.println(result);
      }
}
```

Increment / Decrement operators :

1)pre-increment
2)post-increment
3)pre-decrement
4)post-decrement

++ ➔ increment operator : it increase the value by one.
-- ➔ decrement operator : it reduces the value by one.

Example – 1 :

```
int a = 10;
a++;
System.out.println(a);//11
```

Example – 2 :

```
int a = 10;
a--;
System.out.println(a);//9
```

CodeWithMahi

```
Example – 3 :
int a = 10;
++a;
System.out.println(a);//11

Example – 4 :
int a = 10;
--a;
System.out.println(a);//9


Example – 5 :
int a = 10;
System.out.println(++a);//11

Example – 6 :
int a = 10;
System.out.println(a++);//10
System.out.println(a);//11


Example – 7 :
int a = 10;
System.out.println(--a);//9

Example – 8 :
int a = 10;
System.out.println(a--);//10
System.out.println(a);//9

Example – 9 :
int a = 10;
int b = (++a);
System.out.println(a);//11
System.out.println(b);//11


Example – 10 :
int a = 10;
int b = (a++);
System.out.println(a);//11
System.out.println(b);//10
```

CodeWithMahi

```
Example – 11 :
int a = 10;
int b = (--a);
System.out.println(a);//9
System.out.println(b);//9

Example – 12 :
int a = 10;
int b = (a--);
System.out.println(a);//9
System.out.println(b);//10


Example – 13 :
int a = 10;
a = (++a);
System.out.println(a);//11

Example – 14 :
int a = 10;
a = (--a);
System.out.println(a);//9

Example – 15 :
int a = 10;
a = (a++);
System.out.println(a);//10
```

Note : since we are post-incrementing the variable and storing In the same variable, we will not see the difference in the value, because Assignment will be done before we increment it.
Once after the assignment is done, we cannot increase the same variable in this type of situation.

```
Example – 16 :
int a = 10;
a = (a--);
System.out.println(a);

Example – 17 :
int a = 5;
int b = (++a)+(a++)+(--a)-(a--);
System.out.println(a);//5
System.out.println(b);//12
```

CodeWithMahi

```
Example – 18 :
int a = 5;
int b = (++a)+(++a)+(a++)-(a--);
System.out.println(a);//7
System.out.println(b);//12
```

Combinational operator :

```
Example – 1 :
int a = 10;
a+=5;
System.out.println(a)//15


Example -2 :
int a = 10;
a-=3;
System.out.println(a);//7

Example -3 :
int a = 10;
a*=3;
System.out.println(a);//30


Example -4 :
int a = 10;
a/=2;
System.out.println(a);//5

Example -5 :
int a = 10;
a%=2;
System.out.println(a);//0
```

Bitwise operator :

Bitwise operators perform operations on the bits of the number.

10➔ 1010

Bitwise operators :

1)bitwise AND operator ➔ &
2)bitwise OR operator ➔ |
3)bitwise XOR operator ➔ ^

CodeWithMahi

4)bitwise compliment operator➔ ~
5)bitwise leftshift operator ➔ <<
6)bitwise rightshift operator ➔ >>
7)bitwise unsigned right-shift operator ➔ >>>

1)bitwise AND operator ➔ &

Rule : if both bits are 1, then result is 1.
1 & 1 ➔ 1
1 & 0 ➔ 0
0 & 1 ➔ 0
0 & 0 ➔ 0

```
System.out.println(5 & 7);//5
```

2)bitwise OR operator ➔ |

Rule : if both bits are 0, then result is 0.
1 | 1 ➔ 1
1 | 0 ➔ 1
0 | 1 ➔ 1
0 | 0 ➔ 0

```
System.out.println(5 | 7);//7
```

3)bitwise XOR operator ➔ ^

Rule : if the bits are opposite then 1, else 0.

0 ^ 1 ➔ 1
1 ^ 0 ➔ 1
1 ^ 1 ➔ 0
0 ^ 0 ➔ 0

Q5)WAPT swap the values in two int variables without using extra/third variable.
Example : using extra /third variable

```
package basicsOfJava;

public class IncrementDecrementOp
{
	public static void main(String[] args)
	{
		int a = 5;
```

CodeWithMahi

```
            int b = 7;
            int c = a;
            a = b;
            b = c;
            System.out.println("a value is "+ a);
            System.out.println("b value is "+ b);
      }
}
package basicsOfJava;

public class IncrementDecrementOp
{
      public static void main(String[] args)
      {
            int a = 5;
            int b = 7;
            a = a + b;//12
            b = a - b;//5
            a = a - b;//7
            System.out.println("a value is "+ a);
            System.out.println("b value is "+ b);
      }
}

package basicsOfJava;

public class IncrementDecrementOp
{
      public static void main(String[] args)
      {
            int a = 5;
            int b = 7;
            a = a ^ b;
            b = a ^ b;
            a = a ^ b;
            System.out.println("a value is "+ a);
            System.out.println("b value is "+ b);
      }
}
```

4)bitwise compliment operator➔ ~

1 ➔ 0
0 ➔ 1

CodeWithMahi

It flips every bit.
Rule
1 → 0
0 → 1
Example
5
00000101
After ~
11111010
Java Output
System.out.println(~5);
Output
-6
Why -6?
Formula
~n = -(n + 1)
Example
~5
= -(5+1)
= -6

More examples
~10 = -11
~20 = -21
~100 = -101
Interview question:
What is the shortcut formula for complement?
Answer
~n = -(n+1)
---
5. Left Shift (<<)
Moves bits to the left.
Every left shift multiplies by 2.
Formula
n << k = (n × 2^k)
Example
5 << 1
5 ×2 =10

CodeWithMahi

Binary
5--> 00000101
Shift left-->00001010 =10
System.out.println(5 << 1);
Output : 10

Example
5<<2 -->5×4-->20
Output : 20

6. Right Shift (>>)
Moves bits to the right.

Formula :
n >> k = (n / 2^k)
Example
20>>1-->20/2-->10
Binary-->20-->10100
Shift-->01010-->10
System.out.println(20 >> 1);
Output : 10

Example -->20>>2 -->20/4 -->5
Output : 5
---
7. Unsigned Right Shift (>>>)
This operator always fills the leftmost bits with 0, regardless of the sign.
It is mainly useful with negative numbers.
Example
System.out.println(-10 >> 1);
System.out.println(-10 >>> 1);
Output
-5
2147483643
Interview Point
>> preserves the sign bit.
>>> always inserts 0 on the left.

CodeWithMahi

For positive numbers, >> and >>> produce the same result.

Difference Between >> and >>>

>>	**>>>**
Signed Right Shift	Unsigned Right Shift
Preserves sign	Does not preserve sign
Used for positive and negative numbers	Mainly used for negative numbers
Leftmost bit is copied	Leftmost bit becomes 0

Operator Precedence (High to Low)

~

<< >> >>>

&

^

|

Arithmetic Operators :

1) ()
2) / or % or *
3) + or -

How to read data from user.

1)How to read data from user while executing using command prompt.

1→main method has a String array as argument.
2→ we use this array to pass the values to main method before we execute using command prompt.
3→main method has String array as argument and any data that we pass during execution in command prompt will be stored in String array.

Example code :

```
class MyName

{

      public static void main(String [] a)
```

CodeWithMahi

```
	{

		double x = Double.parseDouble(a[0]);

		double y = Double.parseDouble(a[1]);

		System.out.println("sum is : "+(x+y));

	}

}
```

2)How to read data from user while executing using Scanner object.

Syntax :

1)import the Scanner class from java.util package.

```
	import java.util.Scanner;
```

2)create an object of Scanner class.

```
	Scanner sc = new Scanner(System.in);
```

3)Use in-built methods of Scanner class to read data from console.

```
	byte a = sc.nextByte();
	short a = sc.nextShort();
	int a = sc.nextInt();
	long a = sc.nextLong();
	float a = sc.nextFloat();
	double a = sc.nextDouble();
	String a = sc.next(); or sc.nextLine();
	char a = sc.next().charAt(0);
```

Q6) :

```
package basicsOfJava;

import java.util.Scanner;

public class IncrementDecrementOp
{
	public static void main(String[] args)
	{
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter first number : ");
		int n1 = sc.nextInt();
		System.out.println("Enter second number : ");
		int n2 = sc.nextInt();
		System.out.println(n1+n2);
	}
```

CodeWithMahi

```
}
```

Example -2 :

Q7)WAPTP area of circle. Read radius of a circle from user.

```
package basicsOfJava;

import java.util.Scanner;
public class CircleArea
{
	public static void main(String[] args)
	{
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter radius of a circle : ");
		double r = sc.nextDouble();
		double area = 3.14*r*r;
		System.out.println("Area of circle is : "+area);
	}
}
```

Q8)WAPTP average of three numbers in decimal point number. Read three integers from user.

```
package basicsOfJava;

import java.util.Scanner;

public class AverageOfThree
{
	public static void main(String[] args)
	{
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter three numbers one by one : ");
		int n1 = sc.nextInt();
		int n2 = sc.nextInt();
		int n3 = sc.nextInt();
		double avg = (n1+n2+n3)/3.0;
		System.out.printf("Average of three numbers is : %.2f",avg);
	}
}
```

Q9)WAPTP area of rectangle. Read length and breadth of a rectangle from user.

Input : 12 10

Output : Area of rectangle is : 120

```
package basicsOfJava;
import java.util.Scanner;
```

CodeWithMahi

```
public class AreaOfRectangle
{
	public static void main(String[] args)
	{
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter length and breadth of a rectangle
one by one!!");
		double l = sc.nextDouble();
		double b = sc.nextDouble();
		double area = l*b;
		System.out.println("Area of Rectangle is : "+area);
	}
}
```

Q10)WAPTP simple interest. Read principal, rate of interest and time(years) from user.

Hint : (p*t*r)/100.

```
package basicsOfJava;
import java.util.Scanner;
public class SimpleIntrest
{
	public static void main(String[] args)
	{
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter principal, rate of interest and
time(in years) one by one");
		double p  = sc.nextDouble();
		double r = sc.nextDouble();
		double t = sc.nextDouble();
		double si = (p*t*r)/100;
		System.out.println("Simple interest is : "+si);
	}
}
```

Q11)WAPTPT read name of the user and print it.

```
package basicsOfJava;

import java.util.Scanner;

public class ReadingStringData
{
	public static void main(String[] args)
	{
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter your name : ");
```

CodeWithMahi

```
            String name = sc.nextLine();
            System.out.println("Given name is : "+name);
      }
}
```

Q12)WAPTPT read name of the user and age then print name and age.

Case – 1 : using next();

```
package basicsOfJava;

import java.util.Scanner;

public class ReadingStringData
{
      public static void main(String[] args)
      {
            Scanner sc = new Scanner(System.in);
            System.out.println("Enter your name : ");
            String name = sc.next();
            System.out.println("Enter your age : ");
            int age = sc.nextInt();
            System.out.println("Given name is : "+name);
            System.out.println("Given age is : "+age);
      }
}
```

Output : InputMismatchException

Case-2 : using nextLine();

```
package basicsOfJava;

import java.util.Scanner;

public class ReadingStringData
{
      public static void main(String[] args)
      {
            Scanner sc = new Scanner(System.in);
            System.out.println("Enter your name : ");
            String name = sc.nextLine();
            System.out.println("Enter your age : ");
            int age = sc.nextInt();
            System.out.println("Given name is : "+name);
            System.out.println("Given age is : "+age);
      }
}
```

CodeWithMahi

```
Output :
Virat kohli
38
```

Q13)WAPTP age and name of an user. Read age first then name of the user.

```
package basicsOfJava;

import java.util.Scanner;

public class ReadingStringData
{
	public static void main(String[] args)
	{
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter your age : ");
		int age = sc.nextInt();
		System.out.println("Enter your name : ");
		String name = sc.next();
		System.out.println("Age : "+age);
		System.out.println("Name : "+name);
	}
}
```

```
Enter your age :
38
Enter your name :
Virat kohli
Age : 38
Name : Virat
```

```
package basicsOfJava;

import java.util.Scanner;

public class ReadingStringData
{
	public static void main(String[] args)
	{
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter your age : ");
		int age = sc.nextInt();
		sc.nextLine();
		System.out.println("Enter your name : ");
```

CodeWithMahi

```
            String name = sc.nextLine();
            System.out.println("Age : "+age);
            System.out.println("Name : "+name);
      }
}
```

3)We can read the data from user using IO class Method.
➔IO is a class introduced into java from java 25 onwards.
➔for beginners, scanner object usage will be difficult, so IO class is introduced to make reading the data from user easy.

Important Methods :

➔println(Object obj);

Example :

IO.println("Hello");

Here, data will be printed and cursor will be moved to next line to print next data.

➔print(Object obj);

IO.print("Hello");

Here, data will be printed and cursor will stay in the same line to print next data.

Note : IO.println(); → accepted and cursor will be moved to next line without printing any data

IO.print();→ error. You are neither printing the data nor moving to next line. Hence this statement will be wasted.

➔readln();

This method is used to read the whole line in the string form.

Example – 01 :

```
package basicsOfJava;

import java.util.Scanner;

public class ReadingStringData
{
      public static void main(String[] args)
      {
            System.out.println("Enter your Name : ");
            String name = IO.readln();
            System.out.println("Given name is : "+name);

      }
}
```

CodeWithMani

How to convert String to primitive data.
➔byte a = Byte.parseByte(IO.readln());
➔short a = Short.parseShort(IO.readln());
➔int a = Integer.parseInt(IO.readln());
➔long a = Long.parseLong(IO.readln());
➔float a = Float.parseFloat(IO.readln());
➔double a = Double.parseDouble(IO.readln());
➔String a = IO.readln();
➔char a = IO.readln().charAt(0);

Control Statements :
These are the pre-defined statements used to control the execution of a program.
1)Decision making statements/conditional statements
    1.1)if
    1.2)if-else
    1.3)if-else-if
    1.4)switch
2)Looping statements
    2.1)for
    2.2)while
    2.3)do-while
    2.4)enhanced for {specially used for array and collection}
3)Jumping statements
    3.1)break
    3.2)continue

1.1)if
Syntax :

```
if(CONDITION)
{
    //WRITE CODE
}
```

Q14)WAPTP absolute value of a given integer. Read the integer from user.

```
package basicsOfJava;

import java.util.Scanner;

public class AbsoluteValue
{
```

CodeWithMahi

```
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter an integer : ");
		int n = sc.nextInt();
		if(n<0)
		{
			n = n*-1;
		}
		System.out.println("Absulue value is : "+n);
	}
}
```

Q15)WAPT read the age of the user and great them with
“please welcome!!”.
If the user age is less than 25, great them with
“Hey Young man,
please Welcome!!”

```
package basicsOfJava;

import java.util.Scanner;

public class WelcomeGreeting
{
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter your age : ");
		int age = sc.nextInt();
		if(age<25)
		{
			System.out.println("Hey young man,");
		}
		System.out.println("please Welcome!!");
	}
}
```

1.2)if-else
Syntax :

```
if(condition)
{

}
else
{
```

CodeWithMahi

```
}
```

Q16)WAPTP whether a person is eligible for voting or not. Read age from user.

```
package basicsOfJava;


public class VotingEligiblity
{
	public static void main(String[] args)
	{
		int age = Integer.parseInt(IO.readln("Enter your age : "));
		if(age>=18)
		{
			System.out.println("You are eligible for voting");
		}
		else
		{
			System.out.println("You are not eligible for voting");
		}
	}
}
```

Q17)WAPTP whether given integer is even or odd. Read an integer from user.

```
package basicsOfJava;

public class EvenOrOddProgram
{
	public static void main(String[] args)
	{
		int n = Integer.parseInt(IO.readln("Enter a nunmber : "));
		if(n%2==0)
		{
			IO.println("Even number");
		}
		else
		{
			IO.println("Odd number");
		}
	}
}
```

1.3)if-else-if

Syntax :

```
if(condition)
```

CodeWithMahi

```
{

}
else if(condition)
{

}
else
{

}
```

Q18)WAPTP given number is positive, negative or neutral. Read an integer from user.

```
package basicsOfJava;

import java.util.Scanner;

public class PositiveNegativeNeutral
{
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter a number : ");
		int n = sc.nextInt();
		if(n>0)
		{
			System.out.println("Positive number");
		}
		else if(n<0)
		{
			System.out.println("Negative number");
		}
		else
		{
			System.out.println("Neutral number");
		}
	}
}
```

Q19)WAPTP grade for the obtained marks. Use the below table as reference.

Marks	Grade
90-100	A+
80-89	A
70-79	B

CodeWithMahi

60-69	C
50-59	D
35-49	E
0-34	F
Any other input	Enter valid input

```
package basicsOfJava;

import java.util.Scanner;

public class GradeSystem
{
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter yor marks");
		int marks = sc.nextInt();
		if(marks>=90&&marks<=100)
		{
			System.out.println("A+");
		}
		else if(marks>=80&&marks<=89)
		{
			System.out.println("A");
		}
		else if(marks>=70&&marks<=79)
		{
			System.out.println("B");
		}
		else if(marks>=60&&marks<=69)
		{
			System.out.println("C");
		}
		else if(marks>=50&&marks<=59)
		{
			System.out.println("D");
		}
		else if(marks>=35&&marks<=49)
		{
			System.out.println("E");
		}
		else if(marks>=0&&marks<=34)
		{
			System.out.println("F");
		}
```

CodeWithMahi

```
            else
            {
                  System.out.println("Enter valid input");
            }

      }
}
```

1.4)Switch :

Syntax :

```
      switch(expression)
      {
            case value : {……}
            case value : {……}
            case value : {……}
            case value : {……}
            .
            .
            .
            .
      }
```

Important points on switch :

1)a switch can have any number of cases.
2)only one default case in a switch
3)we can write default case anywhere in the switch.
4)whenever case values doesn't match the expression, then execution starts from default case.
5)Execution always propagates, top to bottom in switch.
6)break is a keyword used to stop the execution and exit from a switch or from a loop.
7)case values need not be continuous.
8)case values can be integers, characters and Strings.
9)case values cannot be decimal point numbers, boolean values.
10) All the case values in a switch must be of same datatype.
11) in java 12, oracle introduced -> token in to switch, later it is confirmed in java 14.
12) some cases with colon and break and some cases with -> without break is no possible.
13)if you use ->, Don't write break, else you will get CTE.
14) we can use block to write code when we have multiple lines.
15) a switch can return a value using the keyword "yield"
16) if a case is having only one line, then don't write yield.
17)if a case is having a block of code(i.e. multiple lines of code) then, writing yield is compulsory.

CodeWithMahi

18) no duplicate case values.
19) in enhanced switch, a case can have multiple case values separated by comma.
20)case values support string from java 7 onwards.
21)if you want to yield any value from a switch, you should write the switch RHS of = operator and end it with semi-colon.
22) one case can yield only one value.

```
package basicsOfJava;

public class SwitchExample
{
	public static void main(String[] args) {
		int a = 2;
		String result = switch(a)
		{
			case 1 ->
			{
				System.out.println("Apple");
				System.out.println("Mango");
				yield "A";
			}
			case 2 -> "B";
			case 3 -> "C";
			case 4 -> "D";
			case 5,6,7,8 -> "E";
			default ->"Enter valid input";
		};

		System.out.println(result);
	}
}
```

Loops :
Loop is a pre-defined statement used to execute the same set of lines repeatedly or again and again until the condition fails.
for loop :
Syntax :

```
for(variable declaration and initialization ; condition; expression)
{
	-------------------------------------------------
	-------------------------------------------------
	-------------------------------------------------

}
```

CodeWithMahi

Q20)WAPTP “Hello” 5 times.
Example 01 :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1  ; i<=5   ;    i++)
		{
			System.out.println("Hello");
		}
	}
}
```

Output :

```
Hello
Hello
Hello
Hello
Hello
```

Example – 02 :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1  ; i>=5   ;    i++)
		{
			System.out.println("Hello");
		}
	}
}
```

Output : No output. Condition is failed in the first iteration only.

Example -03 :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
```

CodeWithMahi

```
            for(int i = 1  ; i<=5   ; i--)
            {
                  System.out.println("Hello");
            }
      }
}
```

Output : infinite times hello will be printed.

Example – 04 :

```
package basicsOfJava;

public class ForLoopDemo
{
      public static void main(String[] args)
      {
            for(int i = 5  ; i>=1   ; i--)
            {
                  System.out.println("Hello");
            }
      }
}
```

Output :

```
Hello
Hello
Hello
Hello
Hello
```

Example 05 :

```
package basicsOfJava;

public class ForLoopDemo
{
      public static void main(String[] args)
      {
            for(int i = 1  ; i<=5   ;          )
            {
                  System.out.println("Hello");
                  i++;
            }
      }
}
```

Output :

```
Hello
Hello
Hello
```

CodeWithMahi

```
Hello
Hello
```

Example – 06 :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1  ; i<=5   ; i++ )
		{
			System.out.println("Hello");
			i++;
		}
	}
}
```

Output :

```
Hello
Hello
Hello
```

Example -07 :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1  ; i<=5   ; i++ )
		{
			System.out.println("Hello");
			i++;
			i++;
			i++;
			i++;
		}
	}
}
```

Output :

```
Hello
```

Example – 08 :

CodeWithMahi

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1  ;i<=5   ; i++ )
		{
			System.out.println("Hello");
			i--;
		}
	}
}
```

Output : infinite times hello

Example – 09 :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1  ;i<=5   ; ++i )
		{
			System.out.println("Hello");
		}
	}
}
```

Output :

```
Hello
Hello
Hello
Hello
Hello
```

Example – 10 :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1  ;i<=5   ; i = i + 2 )
		{
```

CodeWithMahi

```
                System.out.println("Hello");
            }
      }
}

Output :
Hello
Hello
Hello

Example – 11 :
package basicsOfJava;

public class ForLoopDemo
{
      public static void main(String[] args)
      {
            for(int i = 1  ;i<=5 ; i++)
                System.out.println("Hello");
                System.out.println("Bye");
      }
}
Output :
Hello
Hello
Hello
Hello
Hello
Bye
Example – 12 :
package basicsOfJava;

public class ForLoopDemo
{
      public static void main(String[] args)
      {
            for(int i = 1;i<=5;System.out.println("Hello"),i++)
            {
                System.out.println("Bye");
            }
      }
}
Output :
Bye
```

CodeWithMahi

```
Hello
Bye
Hello
Bye
Hello
Bye
Hello
Bye
Hello
```

Example – 13 :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1;i<=5;System.out.println("Hello"),i++);
	}
}
```

Output :

```
Hello
Hello
Hello
Hello
Hello
```

Example – 14 :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		{
			System.out.println("Hello");
		}
	}
}
```

Output : Hello

Example – 15 :

```
package basicsOfJava;
```

CodeWithMahi

```
public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1;i<=5;i++)
		{
			System.out.println("Bye");
		}



		{
			System.out.println("Hello");
		}
	}
}
```

```
Output :
Bye
Bye
Bye
Bye
Bye
Hello
```

```
Example – 16 :
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1;i<=5;System.out.println("Bye"),i++);
		{
			System.out.println("Hello");
		}
	}
}
```

```
Output :
Bye
Bye
```

CodeWithMahi

```
Bye
Bye
Bye
Hello
```

Example – 17 :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1;i<=5;IO.println("Bye"),IO.println("Hii"),i++);
		{
			System.out.println("Hello");
		}
	}
}
```

Output :

```
Bye
Hii
Bye
Hii
Bye
Hii
Bye
Hii
Bye
Hii
Bye
Hii
Hello
```

Example – 17 :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1;i<=5;System.out.println(i),i++);
	}
}
```

Output :

CodeWithMahi

```
1
2
3
4
5
```

Example – 18 :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1;		;i++)
		{
			System.out.println("Hello");
		}
	}
}
```

Output : infinite times Hello

Example -19 :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1; true ;i++)
		{
			System.out.println("Hello");
		}
	}
}
```

Output : infinite times Hello

Example – 20 :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1; false ;i++)
```

CodeWithMahi

```
            {
                  System.out.println("Hello");
            }
      }
}
```

Compile time error .

Example – 21 :

```
package basicsOfJava;

public class ForLoopDemo
{
      public static void main(String[] args)
      {
            int i = 1;
            for(       ; i<=5 ;i++)
            {
                  System.out.println("Hello");
            }
      }
}
```

Output :

```
Hello
Hello
Hello
Hello
Hello
```

Example – 22 :

```
package basicsOfJava;

public class ForLoopDemo
{
      public static void main(String[] args)
      {
            int i = 1;
            for(  int x = 10; i<=5 ;i++)
            {
                  System.out.println("Hello");
            }
      }
}
```

Output :

```
Hello
```

CodeWithMahi

```
Hello
Hello
Hello
Hello
```

23)any variable which is declared inside the for loop is not available to use outside the loop.

24)Any variable declared and initialized inside the loop are not accessible outside the loop.
Example :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		for(int i = 1; i<=5 ;i++)
		{
			int x = 10;
			System.out.println("Hello");
		}
		System.out.println(x);
	}
}
```

Output : CTE

25)any variable declared outside the loop, but initialized inside the loop are not accessible outside the loop.
Example :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		int x;
		for(int i = 1; i<=5 ;i++)
		{
			x = 10;
			System.out.println("Hello");
		}
		System.out.println(x);
```

CodeWithMani

```
	}
}
Output : CTE
```

26)any variable declared outside the loop but initialized inside the loop but before the condition, then we can access the variable outside the loop.

Example :

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		int x;
		int i = 1;
		for(x = 10; i<=5 ;i++)
		{
			System.out.println("Hello");
		}
		System.out.println(x);
	}
}
Output :
Hello
Hello
Hello
Hello
Hello
10
```

27)any variable declared and initialized outside the loop but it is modified or changed inside the loop can be accessed outside the loop with the modified or changed value.

```
package basicsOfJava;

public class ForLoopDemo
{
	public static void main(String[] args)
	{
		int x = 1;
		for(int i = 1; i<=5 ;i++)
		{
			x++;
			System.out.println("Hello");
```

CodeWithMahi

```
            }
            System.out.println(x);
      }
}
```

Q20)WAPTP numbers from 1 to 10 using for loop.

```
package forLoopPrograms;

public class PrintingNumbers1to10
{
      public static void main(String[] args)
      {
            for(int i = 1;i<=10;System.out.println(i),i++);
      }
}
```

Q21)WAPTP numbers from 10 to 1.

```
package forLoopPrograms;

public class PrintingNumbers1to10
{
      public static void main(String[] args)
      {
            for(int i = 10;i>=1;i--)
            {
                  System.out.println(i);
            }
      }
}
```

Q22)WAPTP sum of the numbers from 1 to 5.

```
package forLoopPrograms;

public class PrintingNumbers1to10
{
      public static void main(String[] args)
      {
            int sum = 0;

            for(int i = 1;i<=5;i++)
            {
                  sum = sum + i;
            }

            System.out.println("Sum is : "+sum);
```

CodeWithMahi

```
	}
}
```

Q23)WAPTP product of numbers from 1 to 5.

```
package forLoopPrograms;

public class PrintingNumbers1to10
{
	public static void main(String[] args)
	{
		int product = 1;

		for(int i = 1;i<=5;i++)
		{
			product = product * i;
		}

		System.out.println("product is : "+product);
	}
}
```

Q24)WAPTP numbers in the given range. Read min and max value from user.

```
Input :
23
29
Output :
23
24
25
26
27
28
29
package forLoopPrograms;

public class NumbersInRange
{
	public static void main(String[] args)
	{
		int min = Integer.parseInt(IO.readln("Enter min value"));
		int max = Integer.parseInt(IO.readln("Enter max value"));
		for(int i = min;i<=max;i++)
		{
			System.out.println(i);
		}
```

CodeWithMahi

```
	}
}
```

Q25)WAPTP factorial of a given number. Read the number from user.

```
package forLoopPrograms;

import java.util.Scanner;

public class FactorialOfN
{
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter a number : ");
		int n = sc.nextInt();
		long factorial = 1;
		for(int i = n;i>=1;i--)
		{
			factorial*=i;
		}
		System.out.println("Factorial of "+n+" is : "+factorial);
	}
}
```

Q26)WAPTP factorial of a given number.

```
Factorials of given number are :
1
2
3
6
```

```
package forLoopPrograms;

import java.util.Scanner;

public class FactorsOfN
{
	public static void main(String[] args)
	{
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter a number");
		int n = sc.nextInt();
		for(int i = 1;i<=n; i++)
		{
			//if i divides n, then only print i.
			//agar i ne n ko divide kardiya, thab i ko print karo
```

CodeWithMahi

```
                if(n%i==0)
                {
                      System.out.println(i);
                }
          }
     }
}
```

Q27)WAPTP sum of factors of a given number. Read an integer from user.
Input : 8
Output : 15

```
package forLoopPrograms;

import java.util.Scanner;

public class FactorsOfN
{
     public static void main(String[] args)
     {
          Scanner sc = new Scanner(System.in);
          System.out.println("Enter a number");
          int n = sc.nextInt();
          int sum = 0;
          for(int i = 1;i<=n; i++)
          {
                //if i divides n, then only add i to sum.
                //agar i ne n ko divide kardiya, thab i ko sum se add
karo
                if(n%i==0)
                {
                      sum +=i;
                }
          }
          System.out.println("Sum of factors is : "+sum);
     }
}
```

Q28)WAPTP given number is Perfect number or not. Read an integer from user.
A perfect number is a positive integer that equals the sum of its proper positive divisors (excluding the number itself). For example, 6 is a perfect number because its proper divisors are 1, 2, and 3, and 1 + 2 + 3 = 6. Other well-known perfect numbers include 28, 496, and 8128.

CodeWithMahi

```
package forLoopPrograms;

import java.util.Scanner;

public class PerfectNumber
{
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter a number : ");
		int n = sc.nextInt();
		int sum = 0;
		for(int i = 1;i<=n/2;i++)
		{
			if(n%i==0)
			{
				sum+=i;
			}
		}
		System.out.println("Sum of proper factors is : "+sum);
		if(sum==n)
		{
			System.out.println("Perfect number");
		}
		else
		{
			System.out.println("Not a perfect number");
		}
	}
}
```

Q29)WAPTP count of the factors of a given number. Read an integer from user.
Input : 6
Output : 4

```
package forLoopPrograms;

import java.util.Scanner;
public class CountOfFactors
{
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter a number : ");
		int n = sc.nextInt();
```

CodeWithMahi

```
		int count = 0;
		for(int i = 1;i<=n;i++)
		{
			if(n%i==0)
			{
				count++;
			}
		}

		System.out.println("Count of factors is : "+count);
	}
}
```

Q30)WAPTP given number is prime number or not. Read an integer from user.
If any number is having exactly 2 factors(one and itself) is called prime number.

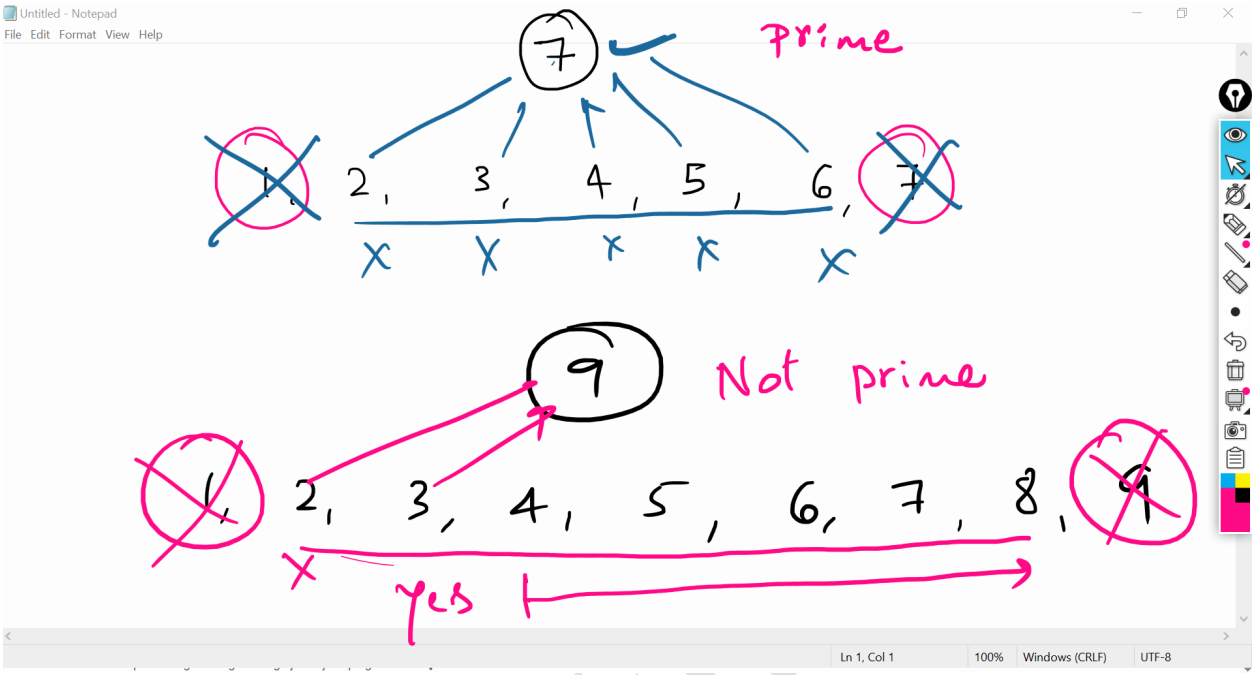
```
package forLoopPrograms;

import java.util.Scanner;
public class CountOfFactors
{
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter a number : ");
		int n = sc.nextInt();
		int count = 0;
		for(int i = 1;i<=n;i++)
		{
			if(n%i==0)
			{
				count++;
			}
		}

		System.out.println("Count of factors is : "+count);
		if(count==2)
		{
			System.out.println("Prime number");
		}
		else
		{
			System.out.println("Not a prime number");
		}
	}
```

CodeWithMani

}




```java
package forLoopPrograms;

import java.util.Scanner;

public class PrimeNumber
{
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter a number : ");
		int n = sc.nextInt();
		boolean isNPrime = true;
		for(int i = 2;i<n;i++)
		{
			if(n%i==0)
			{
				isNPrime = false;
				break;
			}
		}
		if(isNPrime && n>1)
		{
			System.out.println("Prime number");
		}
```

CodeWithMahendra

```
            else
            {
                  System.out.println("Not a prime number");
            }
      }
}
```

Q31)WAPTP all the prime numbers from 1 to 100.
Output :
2
3
5
7
11
13
.
.
.
.
97

```
package forLoopPrograms;

public class PrimeNumbersInRange
{
      public static void main(String[] args)
      {
            //take a loop to traverse from 1 to 100
            for(int n = 1;n<=100;n++)
            {
                  //step-01 : count factors of n
                  int count = 0;
                  for(int i = 1;i<=n;i++)
                  {
                        if(n%i==0)
                        {
                              count++;
                        }
                  }
                  //step-02 : if count is equals to 2, then print n
                  if(count==2)
                  {
                        System.out.println(n);
                  }
            }
```

CodeWithMahi

```
	}
}
```

Q32)break statement :
Example :

```
package forLoopPrograms;

public class BreakKeyWordInLoop
{
	public static void main(String[] args)
	{
		prabhas :
		for(int i = 1;i<=5;i++)
		{
			yash :
			for(int j = 1;j<=5;j++)
			{
				System.out.println("Hello");
				break prabhas;
			}
			System.out.println("Hi");
		}
		System.out.println("Bye");
	}
}
```

While loop :
It is a pre-defined statement used to execute set of lines repeatedly.
Syntax :

```
Variable declaration and initialization;
while(condition)
{
	--------------------
	Operations;
	---------------------

	Update;
}
```

Q33)WAPTP hello 5 times using while loop

```
package whileLoop;

public class WhileDemo
{
	public static void main(String[] args)
	{
```

CodeWithMahi

```
//          for(int i = 1;i<=5;i++)
//          {
//                System.out.println("Hello");
//          }
//          System.out.println(i);
            int i = 1;

            while(i<=5)
            {
                  System.out.println("Hello");
                  i++;
            }
            System.out.println(i);
      }
}
```

Q34)WAPTP sum of the digits of a given number. Read an integer from user.

Input : 345
Output : 12

Logic :

```
            int sum = 0;
            Loop(num!=0)
            {
                  1. take last digit from number
                  2. add the last digit to sum
                  3. remove the last digit from number
            }
            Print the sum
```

```
package whileLoop;

import java.util.Scanner;

public class SumOfDigits
{
      public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);
            System.out.println("Enter a number");
            int num = sc.nextInt();
            int sum = 0;
            while(num!=0)
            {
```

CodeWithMahi

```
                int ld = num%10;
                sum = sum + ld;
                num = num/10;
            }
            System.out.println("Sum of the digits is : "+sum);
      }
}
```

Q35)WAPTP product of the digits of a given number. Read an integer from user.

Input : 345
Output : 60

Logic :

```
            int product = 1;
            Loop(num!=0)
            {
                  1. take last digit from number
                  2.multiply last digit with product
                  3. remove the last digit from number
            }
            Print the product
```

```
package whileLoop;

import java.util.Scanner;

public class ProductOfDigits
{
      public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);
            System.out.println("Enter a number");
            int num = sc.nextInt();
            int product = 1;
            while(num!=0)
            {
                  int ld = num%10;
                  product = product * ld;
                  num = num/10;
            }
            System.out.println("product of the digits is : "+product);
      }
}
```

CodeWithMahi

Q36)WAPTP given number is Spy number or not. Read an integer from user.
A spy number is a number where the sum of its individual digits equals the product of those same digits, such as 1124, 123, and 132.

```
package whileLoop;

import java.util.Scanner;

public class SpyNumber
{
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter a number");
		int num = sc.nextInt();
		int sum = 0;
		int product = 1;
		while(num!=0)
		{
			int ld = num%10;
			sum = sum + ld;
			product = product * ld;
			num = num/10;
		}
		if(sum==product)
		{
			System.out.println("Spy number");
		}
		else
		{
			System.out.println("Not a spy  number");
		}
	}
}
```

Q37)WAPTP given number is Duck number or not. Read an integer from user.

A duck number is a positive number that contains at least one zero, but does not start with a zero. Examples include 3210, 70709, and 102. Numbers like 035 or 02364 are not duck numbers because they start with zero.

```
package whileLoop;
```

CodeWithMahi

```
import java.util.Scanner;

public class DuckNumber
{
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter a number");
		int num = sc.nextInt();
		int product = 1;
		while(num!=0)
		{
			int ld = num%10;
			product = product * ld;
			num = num/10;
		}
		if(product==0)
		{
			System.out.println("Duck number");
		}
		else
		{
			System.out.println("Not a Duck  number");
		}
	}
}
```

Q38)WAPTP reverse the given number. Read an integer from user.

```
package whileLoop;

import java.util.Scanner;

public class SumOfDigits
{
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter a number");
		int num = sc.nextInt();
		int rev = 0;
		while(num!=0)
		{
			int ld = num%10;
			rev = rev*10 + ld;
			num = num/10;
		}
		System.out.println(rev);
```

CodeWithMahi

```
	}
}
```

Q39)WAPTP given number is Palindrome or not. Read an integer from user.

```
package whileLoop;

import java.util.Scanner;

public class PalidromenUMBER
{
	public static void main(String[] args)
	{
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter min range: ");
		int min = sc.nextInt();
		System.out.println("Enter max range: ");
		int max = sc.nextInt();
		for(int num = min;num<=max;num++)
		{
			int temp = num;
			//step-01 : reverse the temp value.
			int rev = 0;
			while(temp!=0)
			{
				int ld = temp%10;
				rev = rev*10 + ld;
				temp = temp/10;
			}

			//step-02 : if rev and num are equal, then print num
			if(rev==num)
			{
				System.out.println(num);
			}
		}
	}
}
```

Q40)WAPTP given number is Strong number or not.
A Strong Number is a special number whose value equals the sum of the factorials of its individual digits. For example, 145 is a strong number because 1! + 4! + 5! = 1 + 24 + 120 = 145.

```
package whileLoop;
```

CodeWithMahi

```java
import java.util.Scanner;

public class StrongNumber
{
	public static void main(String[] args)
	{
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter a number : ");
		int num = sc.nextInt();
		int temp = num;
		int sum = 0;
		while(temp!=0)
		{
			//step-01 : take ld from temp
			int ld = temp % 10;
			//step-02 : find factorial of ld
			int factorial = 1;
			for(int i = ld;i>=1;i--)
			{
				factorial*=i;
			}
			//step-03 : add factorial to sum
			sum = sum + factorial;
			//step-04 : remove ld from temp
			temp = temp / 10;
		}

		if(sum==num)
		{
			System.out.println("Strong number");
		}
		else
		{
			System.out.println("Not Strong number");
		}
	}
}
```

Q41)WAPTP duck numbers in range.

```java
package whileLoop;

import java.util.Scanner;

public class DuckNumberInRange
```

CodeWithMahi

```
{
	public static void main(String[] args) {
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter min range : ");
		int min = sc.nextInt();
		System.out.println("Enter max range : ");
		int max = sc.nextInt();
		for(int num = min;num<=max;num++)
		{
			int temp = num;
			//step-01 : find product of digits of temp
			int product = 1;
			while(temp!=0)
			{
				int ld = temp%10;
				product = product * ld;
				temp = temp / 10;
			}
			//step-02 : if product is zero, then print num.
			if(product==0)
			{
				System.out.println(num);
			}
		}
	}
}
```

Q42) WAPTP given number is Armstrong number or not. Read an integer from user.

An Armstrong number (also called a narcissistic number) is a positive integer equal to the sum of its own digits, each raised to the power of the total number of digits. For instance, 153 is an Armstrong number because it has 3 digits, and $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$

```
package whileLoop;

import java.util.Scanner;

public class ArmstrongNumber
{
	public static void main(String[] args)
	{
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter a number : ");
		int num = sc.nextInt();
```

CodeWithMahi

```
		int temp = num;
		//step-01 : count the number of digits using temp.
		int count = 0;
		while(temp!=0)
		{
			count++;
			temp = temp / 10;
		}
		temp = num;
		//step - 2 : find sum of the digits with power of count
		double sum = 0;
		while(temp!=0)
		{
			//step - 2.1 : take ld from temp
			int ld = temp % 10;
			//step - 2.2 : find ld to the power count.
			double power = Math.pow(ld,count);
			//step - 2.3 : add power to sum
			sum = sum + power;
			//step - 2.4 : remove ld from temp
			temp = temp / 10;
		}

		if(sum==num)
		{
			System.out.println("Armstrong number");
		}
		else
		{
			System.out.println("Not an Armstrong number");
		}
	}
}
```

Q43)WAPTP first n terms of a Fibonacci series. Take n value from user.
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144,……

```
package forLoopPrograms;

import java.util.Scanner;

public class FibonacciSeries
{
	public static void main(String[] args)
	{
```

CodeWithMahi

```
            Scanner sc = new Scanner(System.in);
            System.out.println("Enter n value : ");
            int n = sc.nextInt();
            int a = 0;
            int b = 1;
            int sum = 0;
            for(int i = 1;i<=n;i++)
            {
                  System.out.println(sum);
                  a = b;
                  b = sum;
                  sum = a + b;
            }
      }
}
```

Q44)WAPTP LCM of given two numbers. Read two integers from user.

```
package forLoopPrograms;

import java.util.Scanner;

public class LCMofTwoNumbers
{
      public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);
            System.out.println("Enter two numbers one by one : ");
            int n1 = sc.nextInt();
            int n2 = sc.nextInt();
            for(int i = 1; true ; i++)
            {
                  if((n1*i)%n2==0)
                  {
                        System.out.println("LCM is : "+(n1*i));
                        break;
                  }
            }
      }
}
```

Q45)WAPTP HCF of given two numbers. Read two numbers from user.

```
package forLoopPrograms;

import java.util.Scanner;

public class HCFofTwoNumbers
{
```

CodeWithMahi

```
	public static void main(String[] args)
	{
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter two numbers one by one : ");
		int n1 = sc.nextInt();
		int n2 = sc.nextInt();

		for(int i = n1; i>=1 ; i--)
		{
			if(n1%i==0 && n2%i==0)
			{
				System.out.println("HCF is : "+i);
				break;
			}
		}
	}
}
```

Patterns :

Q46)

```
 *  *  *
 *  *  *
 *  *  *
```

```
package patternPrograms;

public class PatternDemo
{
	public static void main(String[] args)
	{
		for(int i = 1; i<=3; i++)
		{
			for(int j = 1;j<=3;j++)
			{
				System.out.print(" * ");
			}
			System.out.println();
		}
	}
}
```

CodeWithMahi

Q47)

```
 *   *   *   *   *
 *   *   *   *   *
 *   *   *   *   *
 *   *   *   *   *
```

```
package patternPrograms;

public class PatternDemo
{
	public static void main(String[] args)
	{
		for(int i = 1; i<=4; i++)//rows
		{
			for(int j = 1;j<=5;j++)//coloms
			{
				System.out.print(" * ");
			}
			System.out.println();
		}
	}
}
```

Q48)

*	*	*	*	*
*	*	*	*	*
*	@	*	*	*
*	*	*	*	*
*	*	*	*	*

```
package patternPrograms;

public class PatternDemo
{
	public static void main(String[] args)
	{
		for(int i = 1; i<=5; i++)//rows
		{
			for(int j = 1;j<=5;j++)//coloms
			{
				if(i==3&&j==2)
				{
					System.out.print(" @ ");
```

CodeWithMahi

```
                        }
                        else
                        {
                              System.out.print(" * ");
                        }
                  }
                  System.out.println();
            }
      }
}
```

Q49)

*	*	*	*	*
*	*	*	$	*
*	*	*	*	*
*	*	#	*	*
*	*	*	*	*

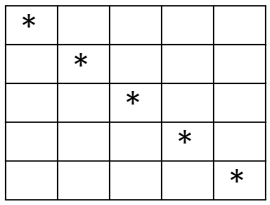
```
package patternPrograms;

public class PatternDemo
{
      public static void main(String[] args)
      {
            for(int i = 1; i<=5; i++)//rows
            {
                  for(int j = 1;j<=5;j++)//coloms
                  {
                        if(i==2&&j==4)
                        {
                              System.out.print(" $ ");
                        }
                        else if(i==4 && j==3)
                        {
                              System.out.print(" # ");
                        }
                        else
                        {
                              System.out.print(" * ");
                        }
                  }
                  System.out.println();
            }
      }
```

CodeWithMahi

```
}
```

Q50)



```
package patternPrograms;

public class PatternDemo
{
	public static void main(String[] args)
	{
		for(int i = 1; i<=5; i++)//rows
		{
			for(int j = 1;j<=5;j++)//coloms
			{
				if(i==j)
				{
					System.out.print(" * ");
				}
				else
				{
					System.out.print("   ");
				}
			}
			System.out.println();
		}
	}
}
```

Q51)

```
package forLoopPrograms;

public class GoldBachNumber
{
	public static void main(String[] args) {
		int num  = 30;
		for(int n = 2;n<=num/2;n++)
		{
			int first = n;
```

CodeWithMahi

```
                int second = num-n;
                //count the factors of first number
                int count1 = 0;
                for(int i = 1;i<=first;i++)
                {
                      if(first%i==0)
                      {
                            count1++;
                      }
                }
                //count the factors of seond number
                int count2 = 0;
                for(int i = 1;i<=second;i++)
                {
                      if(second%i==0)
                      {
                            count2++;
                      }
                }

                if(count1==2&&count2==2)
                {
                      System.out.println(first+", "+second);
                }

          }
     }
}
```

Q52)

				*
			*	
		*		
	*			
*				

```
package patternPrograms;

public class Qsn52
{
      public static void main(String[] args) {
            for(int i = 1;i<=5;i++)
            {
                  for(int j = 1;j<=5;j++)
```

CodeWithMahi

```
                {
                      if(i+j==6)
                      {
                            System.out.print(" * ");
                      }
                      else
                      {
                            System.out.print("   ");
                      }
                }
                System.out.println();
          }
     }
}
```

Logic – 2 :

```
package patternPrograms;

public class Qsn52
{
     public static void main(String[] args) {
          for(int i = 1;i<=5;i++)
          {
                for(int j = 5;j>=1;j--)
                {
                      if(i==j)
                      {
                            System.out.print(" * ");
                      }
                      else
                      {
                            System.out.print("   ");
                      }
                }
                System.out.println();
          }
     }
}
```

Q53)

*				*
	*		*	
		*		
	*		*	

CodeWithMahi

*				*

```
package patternPrograms;

public class Qsn52
{
	public static void main(String[] args) {
		for(int i = 1;i<=5;i++)
		{
			for(int j = 1;j<=5;j++)
			{
				if(i==j || i+j==6)
				{
					System.out.print(" * ");
				}
				else
				{
					System.out.print("   ");
				}
			}
			System.out.println();
		}
	}
}
```

Q54)

*	*	*	*	*
	*			
		*		
			*	
*	*	*	*	*

```
package patternPrograms;

public class Qsn52
{
	public static void main(String[] args) {
		for(int i = 1;i<=5;i++)
		{
			for(int j = 1;j<=5;j++)
			{
				if(i==1 ||i==5 ||i==j)
				{
					System.out.print(" * ");
				}
				else
```

CodeWithMahi

```
                        {
                              System.out.print("    ");
                        }
                  }
                  System.out.println();
            }
      }
}
```

Q55)

*	*	*	*	*
			*	
		*		
	*			
*	*	*	*	*

```
package patternPrograms;

public class Qsn52
{
      public static void main(String[] args) {
            for(int i = 1;i<=5;i++)
            {
                  for(int j = 1;j<=5;j++)
                  {
                        if(i==1 ||i==5 ||i+j==6)
                        {
                              System.out.print(" * ");
                        }
                        else
                        {
                              System.out.print("    ");
                        }
                  }
                  System.out.println();
            }
      }
}
```

Q56)

*	*	*	*	*
*				*
*				*
*				*
*	*	*	*	*

CodeWithMahi

```
package patternPrograms;

public class Qsn52
{
	public static void main(String[] args) {
		for(int i = 1;i<=5;i++)
		{
			for(int j = 1;j<=5;j++)
			{
				if(i==1 ||i==5 ||j==1 || j==5)
				{
					System.out.print(" * ");
				}
				else
				{
					System.out.print("   ");
				}
			}
			System.out.println();
		}
	}
}
```

Q57)

1	1	1	1	1
2	2	2	2	2
3	3	3	3	3
4	4	4	4	4
5	5	5	5	5

```
package patternPrograms;

public class Qsn52
{
	public static void main(String[] args) {
		for(int i = 1;i<=5;i++)
		{
			for(int j = 1;j<=5;j++)
			{
				System.out.print(" "+i+" ");
			}
			System.out.println();
		}
	}
```

CodeWithMahi

```
}
```

Q58)

5	5	5	5	5
4	4	4	4	4
3	3	3	3	3
2	2	2	2	2
1	1	1	1	1

```
package patternPrograms;

public class Qsn52
{
	public static void main(String[] args) {
		for(int i = 5;i>=1;i--)
		{
			for(int j = 1;j<=5;j++)
			{
				System.out.print(" "+i+" ");
			}
			System.out.println();
		}
	}
}
```

Q59)

1	2	3	4	5
1	2	3	4	5
1	2	3	4	5
1	2	3	4	5
1	2	3	4	5

```
package patternPrograms;

public class Qsn52
{
	public static void main(String[] args) {
		for(int i = 1;i<=5;i++)
		{
			for(int j = 1;j<=5;j++)
			{
				System.out.print(" "+j+" ");
			}
			System.out.println();
		}
	}
```

CodeWithMahi

```
}
```

Q60)

5	4	3	2	1
5	4	3	2	1
5	4	3	2	1
5	4	3	2	1
5	4	3	2	1

```
package patternPrograms;

public class Qsn52
{
	public static void main(String[] args) {
		for(int i = 1;i<=5;i++)
		{
			for(int j = 5;j>=1;j--)
			{
				System.out.print(" "+j+" ");
			}
			System.out.println();
		}
	}
}
```

Q61)

10	11	12	13	14
15	16	17	18	19
20	21	22	23	24
25	26	27	28	29
30	31	32	33	34

```
package patternPrograms;

public class Qsn52
{
	public static void main(String[] args)
	{
		int x = 10;
		for(int i = 1;i<=5;i++)
		{
			for(int j = 1;j<=5;j++)
			{
				System.out.print(" "+x+" ");
				x++;
```

CodeWithMahi

```
                }
                System.out.println();
            }
        }
}
```

Q62)

A	B	C	D	E
F	G	H	I	J
K	L	M	N	O
P	Q	R	S	T
U	V	W	X	Y

```
package patternPrograms;

public class Qsn52
{
	public static void main(String[] args)
	{
		char x = 'A';
		for(int i = 1;i<=5;i++)
		{
			for(int j = 1;j<=5;j++)
			{
				System.out.print(" "+x+" ");
				x++;
			}
			System.out.println();
		}
	}
}
```

Q63)

*				
*	*			
*	*	*		
*	*	*	*	
*	*	*	*	*

```
package patternPrograms;

public class Qsn52
{
	public static void main(String[] args)
	{
		for(int i = 1;i<=5;i++)
		{
```

CodeWithMahi

```
                for(int j = 1;j<=5;j++)
                {
                      if(i>=j)
                      {
                            System.out.print(" * ");
                      }
                }
                System.out.println();
          }
     }
}
```

Q64)

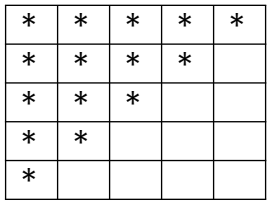
*	*	*	*	*
	*	*	*	*
		*	*	*
			*	*
				*

```
package patternPrograms;

public class Qsn52
{
      public static void main(String[] args)
      {
            for(int i = 1;i<=5;i++)
            {
                  for(int j = 1;j<=5;j++)
                  {
                        if(i<=j)
                        {
                              System.out.print(" * ");
                        }
                        else
                        {
                              System.out.print("   ");
                        }
                  }
                  System.out.println();
            }
      }
}
```

CodeWithMahi

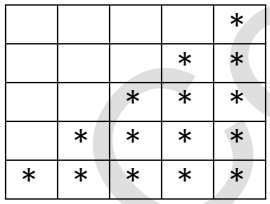
Q65)



```
package patternPrograms;

public class Qsn52
{
	public static void main(String[] args)
	{
		for(int i = 1;i<=5;i++)
		{
			for(int j = 5;j>=1;j--)
			{
				if(i<=j)
				{
					System.out.print(" * ");
				}
			}
			System.out.println();
		}
	}
}
```

Q66)



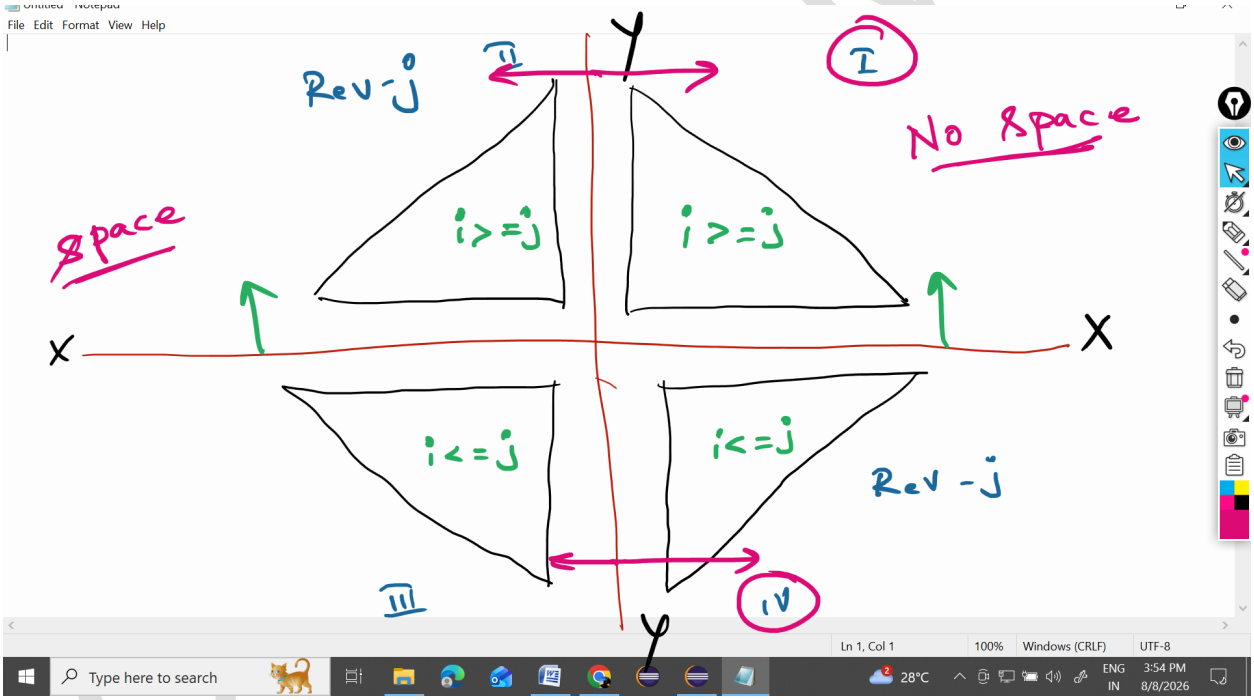
```
package patternPrograms;

public class Qsn52
{
	public static void main(String[] args)
	{
		for(int i = 1;i<=5;i++)
		{
			for(int j = 5;j>=1;j--)
```

CodeWithMahi

```
                {
                        if(i>=j)
                        {
                                System.out.print(" * ");
                        }
                        else
                        {
                                System.out.print("   ");
                        }
                }
                System.out.println();
        }
    }
}
```

Shortcut :



Q67)

				*				
			*	*	*			
		*	*	*	*	*		
	*	*	*	*	*	*	*	
*	*	*	*	*	*	*	*	*

```
package patternPrograms;

public class Pattern02
{
	public static void main(String[] args)
	{
		int space = 4;
		int star = 1;
		for(int i = 1;i<=5;i++)
		{
			//take a loop to print space
			for(int a = 1;a<=space;a++)
			{
				System.out.print("   ");
			}
			//take a loop to print star
			for(int b = 1;b<=star;b++)
			{
				System.out.print(" * ");
			}
			space--;
			star+=2;
			System.out.println();
		}
	}
}
```

Q68)

*	*	*	*	*	*	*	*	*
	*	*	*	*	*	*	*	
		*	*	*	*	*		
			*	*	*			
				*				

```
package patternPrograms;

public class Pattern02
{
	public static void main(String[] args)
	{
		int space = 0;
		int star = 9;
		for(int i = 1;i<=5;i++)
		{
```

CodeWithMahi

```
                //take a loop to print space
                for(int a = 1;a<=space;a++)
                {
                      System.out.print("   ");
                }
                //take a loop to print star
                for(int b = 1;b<=star;b++)
                {
                      System.out.print(" * ");
                }
                space++;
                star-=2;
                System.out.println();
          }
     }
}
```

Q69)

*				
*	*			
*	*	*		
*	*	*	*	
*	*	*	*	*
*	*	*	*	
*	*	*		
*	*			
*				

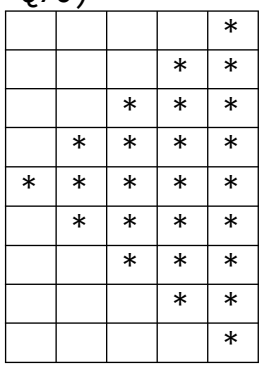
```
package patternPrograms;

public class Pattern02
{
     public static void main(String[] args)
     {
          int star = 1;
          for(int i = 1;i<=9;i++)
          {
                //take a loop to print stars
                for(int a = 1;a<=star;a++)
                {
                      System.out.print(" * ");
                }
                if(i<=4)
                {
```

CodeWithMahi

```
                        star++;
                    }
                    else
                    {
                        star--;
                    }
                    System.out.println();
                }
            }
}
```

Q70)



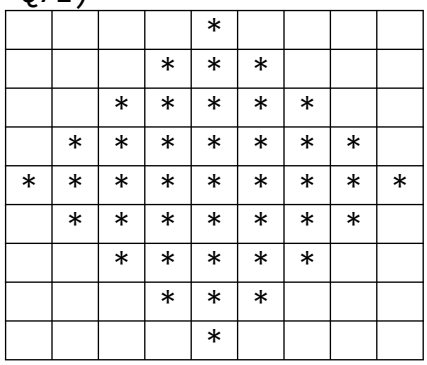
```
package patternPrograms;

public class Pattern02
{
    public static void main(String[] args)
    {
        int space = 4;
        int star = 1;
        for(int i = 1;i<=9;i++)
        {
            //take a loop to print space
            for(int b = 1;b<=space;b++)
            {
                System.out.print("   ");
            }
            //take a loop to print stars
            for(int a = 1;a<=star;a++)
            {
                System.out.print(" * ");
            }
            if(i<=4)
            {
                space--;
                star++;
```

CodeWithMahi

```
                }
                else
                {
                        space++;
                        star--;
                }
                System.out.println();
            }
        }
}
```

Q71)



```
package patternPrograms;

public class Pattern02
{
      public static void main(String[] args)
      {
            int space = 4;
            int star = 1;
            for(int i = 1;i<=9;i++)
            {
                  //take a loop to print space
                  for(int b = 1;b<=space;b++)
                  {
                        System.out.print("   ");
                  }
                  //take a loop to print stars
                  for(int a = 1;a<=star;a++)
                  {
                        System.out.print(" * ");
                  }
                  if(i<=4)
                  {
```

CodeWithMahi

```
                        space--;
                        star+=2;
                  }
                  else
                  {
                        space++;
                        star-=2;
                  }
                  System.out.println();
            }
      }
}
```

Q72)

*	*	*	*	*	*	*	*	*
	*	*	*	*	*	*	*	
		*	*	*	*	*		
			*	*	*			
				*				
			*	*	*			
		*	*	*	*	*		
	*	*	*	*	*	*	*	
*	*	*	*	*	*	*	*	*

```
package patternPrograms;

public class Pattern02
{
      public static void main(String[] args)
      {
            int space = 0;
            int star = 9;
            for(int i = 1;i<=9;i++)
            {
                  //take a loop to print space
                  for(int b = 1;b<=space;b++)
                  {
                        System.out.print("    ");
                  }
```

CodeWithMahi

```
                    //take a loop to print stars
                    for(int a = 1;a<=star;a++)
                    {
                          System.out.print(" * ");
                    }
                    if(i<=4)
                    {
                          space++;
                          star-=2;
                    }
                    else
                    {
                          space--;
                          star+=2;
                    }
                    System.out.println();
              }
        }
}
```

Q73)

*									*
*	*							*	*
*	*	*					*	*	*
*	*	*	*			*	*	*	*
*	*	*	*	*	*	*	*	*	*
*	*	*	*			*	*	*	*
*	*	*					*	*	*
*	*							*	*
*									*

```
package patternPrograms;

public class Pattern02
{
      public static void main(String[] args)
      {
            int star = 1;
            int space = 8;
            for(int i = 1;i<=9;i++)
            {
                  //take a loop to print left side stars
                  for(int a = 1;a<=star;a++)
                  {
                        System.out.print(" * ");
```

CodeWithMahi

```
                }
                //take a loop to print midle spaces
                for(int b = 1;b<=space;b++)
                {
                      System.out.print("   ");
                }
                //take another loop to print right side stars
                for(int c = 1;c<=star;c++)
                {
                      System.out.print(" * ");
                }
                if(i<=4)
                {
                      space-=2;
                      star++;
                }
                else
                {
                      space+=2;
                      star--;
                }
                System.out.println();
          }
     }
}
```

Q75)

				*				
			*		*			
		*		*		*		
	*		*		*		*	
*		*		*		*		*

```
package patternPrograms;

public class Pattern02
{
      public static void main(String[] args)
      {
            int space = 4;
            int starSpace = 1;
            for(int i = 1;i<=5;i++)
            {
                  //take a loop to print space
                  for(int a = 1;a<=space;a++)
```

CodeWithMahi

```
                {
                        System.out.print(" ");
                }
                //take a loop to print starSpace
                for(int b = 1;b<=starSpace;b++)
                {
                        System.out.print("* ");
                }
                space--;
                starSpace++;
                System.out.println();
        }
    }
}
```

continue :
it is a keyword used to skip the iterations in a loop.

Example code :

```
package patternPrograms;

public class Pattern02
{
    public static void main(String[] args)
    {
        for(int i = 1;i<=5;i++)
        {
            if(i==3)
            {
                continue;
            }
            System.out.println(i);
        }
    }
}
```

Do while loop :

```
package patternPrograms;

public class Pattern02
{
    public static void main(String[] args)
    {
        int i = 1;
        do
```

CodeWithMahi

```
		{
			System.out.println("Hello");
			i++;
		}while(i<=5);
	}
}
```

➔while loop first checks condition and then executes the code.
➔do-while first executes the code then it checks condition.
➔compulsory do-while loop executes ones even if the condition is false.

Q76)Pascal triangle :

					1					
				1		1				
			1		2		1			
		1		3		3		1		
	1		4		6		4		1	
1		5		10		10		5		1

```
package patternPrograms;

public class Pattern75
{
	public static void main(String[] args)
	{
		int space = 5;
		int starSpace = 1;
		for(int i = 1; i<=6;i++)
		{
			//take a loop to print space
			for(int a = 1;a<=space;a++)
			{
				System.out.print(" ");
			}
			//take a loop to print starSpace
			int x = 1;
			int num =i-1;
			int den = 1;
			for(int b = 1;b<=starSpace;b++)
			{
				System.out.print(x+" ");
				x = (x*num)/den;
				num--;
```

CodeWithMani

```
                    den++;
                }
                space--;
                starSpace++;
                System.out.println();
            }
        }
}
```

Methods in java :

A method is a block of code used to perform specific operations.

Syntax :

```
Access-modifier Access-specifier return-type name(arguments)
{

}
```

Example :

```
public static void main(String [] args)
{

}
```

public : it is a access-modifier.

There are four access-modifiers

1.public
2)protected
3)default
4)private

static : access-Specifier

it is used to mark java component with static/immutable property.

void : it s a keyword used to represent type of the data that a method returns when it is called.

Main : it is a method name. method name follows camel casing.

(String [] args) : arguments list.

➔methods are written inside the class body only.

➔we cannot write a method inside the method.

CodeWithMahi

➔any number of methods can be written in a class.
➔JVM by default, executes only main method.
➔to execute a method(non-main), we have to perform method calling process.
➔asking JVM to execute the mentioned method is called method-calling process.
➔Syntax : Static method : className.methodName();
Non-static method : objectReferenceVariable.methodName();
➔if the static method belongs to same class, then no need to write className.
➔how many times can we call a method?
Any number of times.
➔order of methods doesn't matter.
Example code :

```
package methodsAndVariables;

public class MethodsDemo
{
	public static void run()
	{
		System.out.println("Running is good");
	}
	public static void jump()
	{
		System.out.println("Frog is jumping");
	}
	public static void main(String[] args)
	{
		System.out.println("Hello");
		run();
		jump();
		System.out.println("Byeee");
	}
}
```

Types of methods : [based on who created it]
1)pre-defined methods
2)user-defined methods.
1)pre-defined methods : these methods are provided by oracle. These are already pre-defined in java.
Example :

```
System.out.println(Math.sqrt(25));
```

Here, sqrt() is a method present in Math class. it is already defined in java by oracle.

CodeWithMahi

We just call it and we use it.

2)user-defined methods.
These are defined or created by us.

Types of methods : [based on static keyword]
1)static methods
2)non-static methods

1)static methods : these methods have static key word in method signature.
Static methods are called by className.
2)non-static methods : these methods will not have static keyword in method signature.
Non-static methods are called by objectName.

Types of methods based on return type and arguments :
1)method without return type and without arguments.
2)method without return type and with arguments.
3)method with return type and without arguments.
4)method with return type and with arguments.

method without return type and without arguments.
In these type of methods, return type will be void and method will not have arguments.

```
package methodsAndVariables;

import java.util.Arrays;

public class MethodsDemo
{
	public static void main(String[] args)
	{
		run();
		System.out.println("Bye");
	}
	public static void run()
	{
		System.out.println("Hello");
	}
}
```

2)method without return type and with arguments.
What are arguments?

CodeWithMahi

These are the variables which are declared in the method signature.
➔a method can have any number of arguments.
➔arguments are separated by comma.
➔arguments can be of different datatypes also.
➔when we call a method, we have to pass the values to initialize these arguments.

```
package methodsAndVariables;

import java.util.Arrays;

public class MethodsDemo
{
	public static void main(String[] args)
	{
		System.out.println("Hello");
		sum(23,"Hi");
	}
	public static void sum(int a,String b)
	{
		System.out.println("Sum is : "+(a+b));
	}
}
```

3)method with return type and without arguments.
➔void will be replaced with the datatype of the value that we want to return.
➔a method can return only one value.
➔returned value, we can,
	1)directly print it
	2)store it in a variable
	3)use it in switch
	4)pass as argument to a method
	5)directly use it in condition
➔always return keyword must be written in the last line of the method.
➔

Example :

```
package methodsAndVariables;

import java.util.Arrays;

public class MethodsDemo
```

CodeWithMahi

```
{
	public static void main(String[] args)
	{
		//Step-01 : we can directly print the returned value
		//System.out.println("Pi value is : "+getPi());
		//Step-02 : we can store the returned value into a variable
//		double s = getPi();
//		System.out.println(s);
		//Step-03 : using returned value in switch
//		switch(getPi())
//		{
//			case 1->System.out.println("East");
//			case 2->System.out.println("West");
//			case 3->System.out.println("North");
//			case 4->System.out.println("South");
//			default ->System.out.println("Enter valid input");
//		}
		//Step-04 : we can pass the returned value as argument for
another method also
		//printInt(getPi());
		//Step-05 : we can directly use returned value in conditions
also
//		if(getPi()==4)
//		{
//			System.out.println("Hello");
//		}
//		else
//		{
//			System.out.println("Byee");
//		}

	}
	public static void printInt(int a)
	{
		System.out.println("Given int value is : "+a);
	}
	public static int getPi()
	{
		return 4;
	}
}
```

Example :

```
package methodsAndVariables;
```

CodeWithMahi

```
import java.util.Arrays;

public class MethodsDemo
{
	public static void main(String[] args)
	{
		//getPi();
		System.out.println(getPi());
	}

	public static int getPi()
	{
		System.out.println("Hello");
		int i = 10;
		if(i<2)
		{
			return 4;
		}
		System.out.println("Bye");
		return 0;
	}
}
```

Example :

```
package methodsAndVariables;

public class MethodsDemo
{
	public static void main(String[] args)
	{
		getPi();
	}

	public static void getPi()
	{
		return;
	}
}
```

4)method with return type and with arguments.

```
package methodsAndVariables;
```

CodeWithMahi

```
public class MethodsDemo
{
	public static void main(String[] args)
	{
		System.out.println("Sum is : "+getSum(12, 13));
	}

	public static int getSum(int a, int b)
	{
		return a+b;
	}
}
```

➔Formal arguments and Actual arguments.

Formal arguments/Formal parameters : these are the variables which are declared in the method signature.

Actual arguments/actual parameters : these are the values that we pass during method calling process.

Example :

```
package methodsAndVariables;

public class MethodsDemo
{
	public static void main(String[] args)
	{
		System.out.println("Sum is : "+getSum(12, 13));
	}

	public static int getSum(int a, int b)
	{
		return a+b;
	}
}
```

In the above code, a and b are formal arguments, 12 and 13 are actual arguments.

Why methods ?

```
package methodsAndVariables;

public class MethodsDemo
{
	public static void main(String[] args)
	{
		int num1 = 234,num2 = 217, num3 = 924, num4 = 678;
```

CodeWithMahi

```
            System.out.println("Reverse of 1st number is :
"+reverse(num1));
            System.out.println("Reverse of 2nd number is :
"+reverse(num2));
            System.out.println("Reverse of 3rd number is :
"+reverse(num3));
            System.out.println("Reverse of 4th number is :
"+reverse(num4));
      }
      public static int reverse(int num)
      {
            int rev = 0;
            while(num!=0)
            {
                  rev = rev*10+(num%10);
                  num/=10;
            }
            return rev;
      }
}
```

How to call non-static methods.
➔we can call non-static methods with the object reference.

Syntax to create an object :
ClassName orv = new ClassName();

Then,
Orv.methodName();

Example :

```
package methodsAndVariables;

public class MethodsDemo
{
      public static void main(String[] args)
      {
            MethodsDemo obj = new MethodsDemo();
            int num1 = 234;
            System.out.println(obj.reverse(num1));
      }
      public int reverse(int num)
      {
            int rev = 0;
```

CodeWithMahi

```
            while(num!=0)
            {
                  rev = rev*10+(num%10);
                  num/=10;
            }
            return rev;
      }
}
```

Variables :

Types of variables :

1)Global variables(class – level variables)

1.1)static variables

1.2)non-static variables

2)Local variables

Local variables:

➔These are the variables which are declared inside the method.

➔these variables must be initialized before we use it.

➔default values are not-applicable for local variables.

Syntax of local variable :

Datatype variableName = data;

➔no access_modifiers or access_specifiers are applicable to local variables.

➔one method's local variable cannot be used by another method.

➔parameters are also local variables.

Global variables/Class-level variables :

These variables are declared outside the methods but inside the class that is why these variables are called class-level variables.

➔static variables :

These variables are declared with the keyword static.

➔non-static variables :

These variables are not declared with the keyword static.

➔class-level variables will have default values.

Default values :

int→0

double→0.0

boolean→false

char→empty character

non-primitive data : null

➔Syntax : access_modifier access-specifier datatype variableName = data;

Method overloading :

CodeWithMahi

➔Having same method name with different parameter list is called method overloading.
➔to perform the same operation in different ways we use method overloading.
➔we can overload static methods as well as non-static methods also.

Example :

```
package methodOverloading;

public class MethodOverloading
{
	public static void main(String[] args)
	{
		add(10,20,30);
	}
	public static void add()
	{
		System.out.println("Addition");
	}
	public static void add(int a, int b)
	{
		System.out.println("Sum is : "+(a+b));
	}
	public static void add(int a, int b, int c)
	{
		System.out.println("Sum is : "+(a+b+c));
	}
}
```

Example :

```
package methodOverloading;

public class MethodOverloading
{
	public static void main(String[] args)
	{
		MethodOverloading obj = new MethodOverloading();
		obj.add();
	}
	public void add()
	{
		System.out.println("Addition");
	}
	public void add(int a, int b)
	{
```

CodeWithMahi

```
			System.out.println("Sum is : "+(a+b));
		}
		public void add(int a, int b, int c)
		{
			System.out.println("Sum is : "+(a+b+c));
		}
}
```

Example : 3

```
package methodOverloading;

public class MethodOverloading
{
		public static void main(String[] args)
		{
			add(10,20);
		}
		public static void add(int a, int b)
		{
			System.out.println("Hii");
		}
		public static void add(int a, double b)
		{
			System.out.println("Bye");
		}
		public static void add(double a, int b)
		{
			System.out.println("Hello");
		}
		public static void add(double a, double b)
		{
			System.out.println("Get lost");
		}
}
```

➔static methods can directly access static variables.
Why? Because they both stay in the same Class static area.
➔static methods cannot access non-static variables directly.
Why? Because non-static variables are present in objects(heap area).
Solution? :
Create an object and use that object reference to access the non-static variables.

➔static methods can directly access static methods.

CodeWithMahi

Why? Because they both stay in the same Class static area.
➔static methods cannot access non-static methods directly.
Why? Because non-static methods are present in objects(heap area).
Solution? :
Create an object and use that object reference to access the non-static methods.

```
package methodOverloading;

public class MethodOverloading
{
	public static int k = 100;
	public int j = 200;
	public static void main(String[] args)
	{
		System.out.println(k);
		run();
		MethodOverloading obj = new MethodOverloading();
		System.out.println(obj.j);
		obj.walk();
	}
	public static void run()
	{
		System.out.println("Run fast");
	}
	public void walk()
	{
		System.out.println("walk slowly");
	}
}
```

Steps
1)Three major areas in the execution diagram
1.1)stack area
1.2)heap area
1.3)class static area/Method area
➔class loader enters the stack first.
➔load all static members of the class in which main method is present.
➔class loader exit from the stack.
➔main method is loaded into the stack.
➔main method can access static members directly.
➔main method cannot access non-static members, it has to create object.

CodeWithMahi

➔Syntax :
ClassName orv = new ClassName();
Tasks performed by new keyword.
→creates an object in the heap area.
→load all non-static members of the class to which the object is created.
→return the address of the object.
➔main method can use this object address to access non-static methods or non-static variables.
➔once main method finishes its task, then main method is terminated from stack.
➔at last Garbage collector will enter the stack to clean the all memory areas.

➔static members can directly access static members.
Why?
They both stay is same CSA
➔ non-static members can directly access non-static members.
Why?
They both stay is same object.
➔non-static members can directly access static members.
Why?
Because static members are single-copy nature.
➔static members cannot directly access non-static members.
Why?
Because non-static members are multiple-copy nature.

Multiple classes in java :
➔we can create multiple classes in a file.
➔only one public class should be there in a file.
➔name of the public class should be the name of the file.
➔any number of classes can be created in a file.
➔to access static members of another class, use class name to refer it.
➔ to access non-static members of another class, create object of that respective class and use orv to access it or to refer it.
➔to access static members of same class, No need to use any reference. You can use class name to refer but it is optional.
➔to access the non-static members of same class, you need to create object of the current class.

Class and object :
Class :

CodeWithMani

➔Class is a type-definition block used to define the non-primitive data.
➔class is a blue-print of object.
➔class is a combination of variables and methods.
➔every class is a non-primitive data.
➔we can create any number classes in a file.
➔class name follows Pascal casing.
➔when we have multiple classes in a file, file should be saved with the className which is public.

```
public class A
{

}
class B
{

}
Class C
{

}
```

Answer : A.java
➔in java, a file cannot have more than one public class.

Object :
➔Object is the real entity which is created for the class.
➔object exists in real world.
⇨ Every object will have state and behavior.
State→variables
Behavior → methods.
➔objects are created with new keyword.
Syntax to create object : ClassName orv = new ClassName();
➔we can create any number of objects to a class.

➔A variable should be static if its value is same for all the objects.
➔A variable should be non-static if its value changes from one object to another object.
➔a method should be non-static if it uses any non-static variable. Otherwise the method should be static.

```
package classObject;
```

CodeWithMahi

```
public class Demo
{
	public static void main(String[] args)
	{
		Student s1 = new Student();
		s1.name = "Abhi";
		s1.age = 12;
		s1.school = "ABC school";
		s1.city = "Hyderabad";
		Student s2 = new Student();
		s2.name = "Ramya";
		s2.age = 13;
		s2.school = "ABC school";
		s2.city = "Hyderabad";
		Student s3 = new Student();
		s3.name = "Shekhar";
		s3.age = 15;
		s3.school = "ABC school";
		s3.city = "Hyderabad";
		System.out.println("Student 1 details : ");
		s1.displayDetails();
		System.out.println("Student 2 details : ");
		s2.displayDetails();
		System.out.println("Student 3 details : ");
		s3.displayDetails();
	}
}
class Student
{
	public String name;
	public int age;
	public static String school;
	public static String city;
	public void displayDetails()
	{
		System.out.println("Name : "+name);
		System.out.println("Age : "+age);
		System.out.println("School : "+school);
		System.out.println("City : "+city);
	}
}
```

➔Initialize the static variables at the time of declaration only.

CodeWithMahi

➔don't initialize the non –static variables at the time of declaration.
Why : because, these non-static variable values will change from one object to another object.
➔Will I get error if I initialize non-static variables at the time of declaration only?
NO. you will not get error.

```
package classObject;

public class Demo
{
	public static void main(String[] args)
	{
		Student s1 = new Student();
		s1.name = "Abhi";
		s1.age = 12;
		Student s2 = new Student();
		s2.name = "Ramya";
		s2.age = 13;
		Student s3 = new Student();
		s3.name = "Shekhar";
		s3.age = 15;
		System.out.println("Student 1 details : ");
		s1.displayDetails();
		System.out.println("Student 2 details : ");
		s2.displayDetails();
		System.out.println("Student 3 details : ");
		s3.displayDetails();
	}
}
class Student
{
	public String name;
	public int age;
	public static String school = "ABC school";
	public static String city = "Hyderabad";
	public void displayDetails()
	{
		System.out.println("Name : "+name);
		System.out.println("Age : "+age);
		System.out.println("School : "+school);
		System.out.println("City : "+city);
	}
}
```

CodeWithMahi

Constructor : it is a special type of method used to initialize non-static variables of the class.

How to declare the constructor?

Syntax :

```
Access-Modifier ClssName()
{

}
```

What will be the constructor name?

Answ : same as className.

Can we overload the constructor like methods?

Answer : Yes. We can overload the constructor.

this :

it is a keyword used to represent the current object.

```
package classObject;

public class Demo1
{
	public static void main(String[] args)
	{
		//step-01 : create the object
		Car c1 = new Car();
		//step-02 : initializing non-static variables of the c1 object
		//way-01 : manually like one by one variables are initialized
//		c1.company = "Honda";
//		c1.colour = "Red";
//		c1.milage = 18;

		//way-02 : define a method to initialize non-static variables
		//c1.initialization("Honda", "Red", 18);

		//way-03 : initialize non-static variables with the help of constructor
		c1.displayDetails();
	}
}
class Car
{
```

CodeWithMahi

```
	public String company;
	public String colour;
	public double milage;
	public void initialization(String company, String colour, double
milage)
	{
		this.company = company;
		this.colour  = colour;
		this.milage = milage;
	}
	public void displayDetails()
	{
		System.out.println("Company : "+company);
		System.out.println("Colur : "+colour);
		System.out.println("Milage : "+milage);
	}
}

Example :
package classObject;

public class Demo1
{
	public static void main(String[] args)
	{
		//step-01 : create the object
		//Car c1 = new Car();
		//step-02 : initializing non-static variables of the c1
object

		//way-01 : manually like one by one variables are
initialized
//		c1.company = "Honda";
//		c1.colour = "Red";
//		c1.milage = 18;

		//way-02 : define a method to initialize non-static
variables
		//c1.initialization("Honda", "Red", 18);

		//way-03 : initialize non-static variables with the help of
constructor
		Car c1 = new Car("Honda","Red",18);
		Car c2 = new Car("Tata", "Yellow", 14);
```

CodeWithMahi

```
			c1.displayDetails();
		}
}
class Car
{
		public String company;
		public String colour;
		public double milage;
		public Car(String company, String colour, double milage)
		{
			this.company = company;
			this.colour  = colour;
			this.milage = milage;
		}
//		public void initialization(String company, String colour, double
milage)
//		{
//			this.company = company;
//			this.colour  = colour;
//			this.milage = milage;
//		}
		public void displayDetails()
		{
			System.out.println("Company : "+company);
			System.out.println("Colur : "+colour);
			System.out.println("Milage : "+milage);
		}
}
```

➔every class must have constructor.
➔without constructor, we cannot create object for the class.

Types of constructors :
1)default constructor
2)user-defined constructor

2)user-defined constructor
These constructors are defined by user.

CodeWithMani

1)default constructor
This is the constructor given by compiler at the time of compilation.
➔Compiler will check whether a class has constructor or not. If the class don't have at least one constructor, then it adds a constructor called default constructor.

```
public ClassName()
{
}
```

Example :

```
package classObject;

public class Demo1
{
	public static void main(String[] args)
	{
		Car c1 = new Car("Honda");//error : default constructor don't have arguments
		c1.jump();
	}
}
class Car
{
	public void jump()
	{
		System.out.println("Jump high");
	}
}
```

➔if developer writes at least one constructor also, then compiler will not add any default constructor.
➔constructors are implicitly called at the time of object creation.
➔can we explicitly call the constructor?
No, they are implicitly called at the time of object creation only.
➔a class can have any number of constructors.
➔we can overload the constructor.

Constructor chaining.
Calling one constructor from another constructor is called constructor chaining.
How do you achieve it?
By using `this();`
➔we can call only one constructor from another constructor.

Example – 1:

```
package classObject;
```

CodeWithMahi

```
public class ConstructorCallDemo
{
	public static void main(String[] args)
	{
		new A();
	}
}
class A
{
	public A()
	{
		this(1,3);
		System.out.println("Hello");
	}
	public A(int k)
	{
		System.out.println("Byee");
	}
	public A(int a, int b)
	{
		this(1);
		System.out.println("Hii");
	}
}
```

Output :

```
Byee
Hii
Hello
```

Example – 2 :

```
package classObject;

public class ConstructorChaining
{
	public static void main(String[] args)
	{
		Student01 s1 = new Student01("Abhi",501,21,98.34);
		Student01 s2 = new
Student01("Suma",505,20,64.55,"Hyderabad");
		s1.displayDetails();
		System.out.println("-------------------------------------
");
		s2.displayDetails();
```

```
	}
}
class Student01
{
	public String name;
	public int StudentId;
	public int age;
	public double percentage;
	public String city;
	public Student01(String name,int StudentId,int age,double
percentage)
	{
		this.name = name;
		this.StudentId = StudentId;
		this.age = age;
		this.percentage = percentage;
	}
	public Student01(String name,int StudentId,int age,double
percentage,String city)
	{
		this(name,StudentId,age,percentage);//constructor-call
statement
		this.city = city;
	}
	public void displayDetails()
	{
		System.out.println("name : "+name );
		System.out.println("Student id : "+StudentId);
		System.out.println("Age : "+age);
		System.out.println("Percentage : "+percentage);
		System.out.println("City : "+city);
	}
}
Output :
name : Abhi
Student id : 501
Age : 21
Percentage : 98.34
City : null
-------------------------------------
name : Suma
Student id : 505
Age : 20
Percentage : 64.55
```

```
City : Hyderabad
```

1. Book Class (Basic Object + Method)

Create a class called `Book`

Variables:

- bookName
- authorName
- price

Methods:

- **`displayBookInfo()`** → print all book details
- **`issueBook()`** → print:
  "Book issued successfully: <bookName>"

Tasks:

- Read all values from user in main method
- Create object and initialize values
- Call displayBookInfo().

```
package classObject;

import java.util.Scanner;

public class Qsn01
{
	public static void main(String[] args)
	{
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter bookName, authorName and price one
by one");
		String bookName = sc.nextLine();
		String authorName = sc.nextLine();
		double price = sc.nextDouble();
		Book b1 = new Book(bookName, authorName, price);
		b1.displayBookInfo();
```

eWithMahi

```
			b1.issueBook();
		}
}
class Book
{
		//step-01 : declare static and non-static variables
		public String bookName;
		public String authorName;
		public double price;
		//step-02 : initialize the non-static variables with constructor
		public Book(String bookName, String authorName, double price)
		{
			this.bookName = bookName;
			this.authorName = authorName;
			this.price = price;
		}
		//step-03 : write any methods required.
		public void	displayBookInfo()
		{
			System.out.println("Book name : "+bookName);
			System.out.println("Author name : "+authorName);
			System.out.println("Price : "+price);
		}
		public void issueBook()
		{
			System.out.println("Book issued successfully : "+bookName);
		}
}
```

2. BankAccount Class (Important)

Create a class called **`BankAccount`**

Variables:

- accountHolderName
- accountNumber
- balance

Methods:

- **`deposit(double amount)`** → add amount to balance

- **`withdraw(double amount)`**
  - If balance ≥ amount → deduct
  - Else print: "Insufficient Balance"
- **`displayBalance()`**

Task:

- create a BankAccount object. Read input from user .
- Perform deposit and withdraw
- Display bank balance

```
package classObject;

import java.util.Scanner;

public class Qsn02
{
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.println("Enter AccountHolderName, accountNumber and balance
to create the account");
String accountHolderName = sc.nextLine();
String accountNumber = sc.nextLine();
double balance = sc.nextDouble();
BankAccount acc1 = new BankAccount(accountHolderName, accountNumber,
balance);
acc1.deposit(500);
acc1.withdraw(1000);
acc1.DisplayBalance();
}
}
class BankAccount
{
//step-01 : declare static and non-static variables
public String accountHolderName;
public String accountNumber;
public double balance;
//step-02 : initialize non-static variables with constructor
```

```
public BankAccount(String accountHolderName, String accountNumber,
double balance)
{
this.accountHolderName = accountHolderName;
this.accountNumber = accountNumber;
this.balance = balance;
}
//step-03 : write any methods required.
public void deposit(double amount)
{
balance = balance+amount;
System.out.println("Deposit of "+amount+" is done successfully..!!");
}
public void withdraw(double amount)
{
if(amount>balance)
{
System.out.println("Insufficient balance..!!");
}
else
{
balance = balance - amount;
System.out.println("Amount of "+amount+" is deducted
successfully..!!");
}
}
public void DisplayBalance()
{
System.out.println("Current balance is : "+balance);
}
}
```

3. Student Class (Result System)

Create a class called `Student`

Variables:

- name
- marks1, marks2, marks3

Methods:

- `calculateTotal()` → return total
- `calculateAverage()` → return average
- `displayResult()` → print all details

Task:

- create a student object. Read input from user .
- Display result.

4. Employee Class (Salary Calculation)

Create a class called `Employee`

Variables:

- name
- basicSalary

Methods:

- `calculateHRA()` →return 10% of salary
- `calculateDA()` → return 20% of salary
- `calculateTotalSalary()` → return total salary
- `displaySalaryDetails()`

Task:

- create a EMPLOYEE object. Read input from user .
- Display result.

5. Mobile Class

Create a class called `Mobile`

Variables:

CodeWithMahi

- brand
- price
- batteryPercentage

Methods:

- `chargeBattery(int amount)` → increase battery
- `useMobile(int amount)` → decrease battery
- `displayMobileInfo()`

6. Rectangle Class

Create a class called `Rectangle`

Variables:

- length
- breadth

Methods:

- `calculateArea()`
- `calculatePerimeter()`
- `displayDetails()`

7. Circle Class

Create a class called `Circle`

Variables:

- radius

Methods:

- `calculateArea()`
- `calculateCircumference()`
- `displayCircleInfo()`

CodeWithMahi

8. Patient Class

Create a class called `Patient`

Variables:

- name
- age
- disease

Methods:

- `displayPatientInfo()`
- `checkCritical()`
    - If age > 60 → print "High Risk Patient"

9. Product Class (Billing System)

Create a class called `Product`

Variables:

- productName
- price
- quantity

Methods:

- `calculateBill()` → return total price
- `displayBill()`

10. Vehicle Class

Create a class called `Vehicle`

Variables:

- vehicleName

CodeWithMahi

- fuel

Methods:

- `refuel(int amount)` → increase fuel
- `drive(int amount)` → decrease fuel
- If fuel < required → print "Not enough fuel"
- `displayVehicleInfo()`

Static and non-static blocks.
Static blocks :
➔these are the block of code which is written inside the class but outside the methods.
➔these block are marked with static keyword.
➔these blocks will get executed at the time of loading.
➔we can write any number of static blocks in a class.
➔order of execution of static blocks is top-bottom.
➔static blocks are used often to initialize static variables.
➔static blocks are executed only one time. That is at the time of loading.

Example :

```
package staticNonStaticBlocks;

public class Demo
{
	public static int x = 10;
	public static void main(String[] args)
	{
		System.out.println("Bye");
	}

	public static void run()
	{
		System.out.println("Run fast");
	}
	static
	{
		System.out.println("X value is : "+x);
	}
	static
	{
		System.out.println("X value is : "+x);
```

```
	}
}
Output :
 X value is : 10
X value is : 10
Bye

Example :
package staticNonStaticBlocks;

public class Demo
{
	public static void main(String[] args)
	{
		System.out.println("Bye");
		A ob = new A();
	}
	static
	{
		System.out.println("Hello");
	}
}
class A
{
	public A()
	{
		System.out.println("Good");
	}
	static
	{
		System.out.println("Hii");
	}
}
Hello
Bye
Hii
Good

Example :
package staticNonStaticBlocks;

public class Demo
{
	public static void main(String[] args)
```

```
	{
		System.out.println("Bye");
		A ob1 = new A();
		A ob2 = new A();
		A ob3 = new A();
	}
	static
	{
		System.out.println("Hello");
	}
}
class A
{
	public A()
	{
		System.out.println("Good");
	}
	static
	{
		System.out.println("Hii");
	}
}
```

```
Output :
Hello
Bye
Hii
Good
Good
Good
```

EXAMPLE :

```
package staticNonStaticBlocks;

public class Demo
{
	public Demo()
	{
		System.out.println("Sun");
	}
	public static void main(String[] args)
	{
		System.out.println("Bye");
```

```
		Demo d1 = new Demo();
		Demo d2 = new Demo();
	}

	{
		System.out.println("Hello");
	}
}
//class A
//{
//	public A()
//	{
//		System.out.println("Good");
//	}
//	static
//	{
//		System.out.println("Hii");
//	}
//}
```

OUTPUT :

```
Bye
Hello
Sun
Hello
Sun
```

Example :

```
package staticNonStaticBlocks;

public class Demo
{
	static
	{
		System.out.println("Moon");
	}
	public Demo()
	{
		System.out.println("Sun");
	}
	public static void main(String[] args)
	{
		System.out.println("Bye");
		Demo d1 = new Demo();
		A d2 = new A();
```

```
	}

	{
		System.out.println("Hello");
	}
}
class A
{
	public A()
	{
		System.out.println("Good");
	}
	static
	{
		System.out.println("Hii");
	}

	{
		System.out.println("System");
	}
}
```

```
Output :
Moon
Bye
Hello
Sun
Hii
System
Good
```

Inheritance :

➔It is the process of sharing roperties like methods and variables f one class to another class with the help of **extends** keyword.

➔the class which shares its properties is called super class/parent class.

➔the class which uses the properties of other class is called sub-class/ child class.

➔**every sub-class constructor will implicitly call super-class constructor.**

Example :

```
package staticNonStaticBlocks;

public class Demo
```

```
{
	public Demo()
	{
		System.out.println("Hiii");
	}
	public static void main(String[] args)
	{
		System.out.println("Hello");
		new A();
	}
}
class A extends Demo
{
	public A()
	{
		super();
		System.out.println("Byee");
	}
}
Hello
Hiii
Byee


package staticNonStaticBlocks;

public class Demo
{
	public static void main(String[] args)
	{
		System.out.println("Hello");
		new A();
	}
}
class A extends Demo
{
	public A()
	{
		System.out.println("Byee");
	}
}

Output :
Hello
```

Byee

Only default constructor(non-argumented constructor) will be called implicitly. If super class has argumented constructor, then developer explicitly should call super class constructor with the help of super().

```
package staticNonStaticBlocks;

public class Demo
{
	public Demo(int k)
	{
		System.out.println("Hiii");
	}
	public static void main(String[] args)
	{
		System.out.println("Hello");
		new A();
	}
}
class A extends Demo
{
	public A(int a, int b)
	{
		super(10);
		System.out.println("Good");
	}
	public A()
	{
		System.out.println("Byee");
		super(10);
	}
}
```

Every sub-class constructor must call super class constructor.

Inheritance Example :

```
package inheritanePrograms;

public class Main
{
	public static void main(String[] args)
	{
		Addition add = new Addition(10,20);
```

```
            add.getSum();
            Multiplication mul = new Multiplication(2, 5);
            mul.getProduct();

      }
}
class Calculation
{
      public int x;
      public int y;
      public Calculation(int x, int y)
      {
            this.x = x;
            this.y = y;
      }

}
class Addition extends Calculation
{
      public Addition(int x, int y)
      {
            super(x,y);
      }
      public void getSum()
      {
            System.out.println("Sum is : "+(x+y));
      }
}
class Subtraction extends Calculation
{
      public Subtraction(int x, int y)
      {
            super(x,y);
      }
      public void getDifference()
      {
            System.out.println("Difference is : "+(x-y));
      }
}
class Multiplication extends Calculation
{
      public Multiplication(int x, int y)
      {
            super(x,y);
```

```
	}
	public void getProduct()
	{
		System.out.println("Product is : "+(x*y));
	}
}
class Division extends Calculation
{
	public Division(int x, int y)
	{
		super(x,y);
	}
	public void getDivision()
	{
		System.out.println("Quotient is : "+(x/y));
	}
}
class Remainder extends Calculation
{
	public Remainder(int x, int y)
	{
		super(x,y);
	}
	public void getRemainder()
	{
		System.out.println("Remainder is : "+(x%y));
	}
}
```

CodeWith

CodeWithMahi

CodeWithMahi